arXiv:2611.13960v1 [cs.CR] 10 Nov 2026

From Attack Surface to Defense Architecture: Compositional Security for Agentic Coding Harnesses

Simon Hegelich [1] **Bardh Prenkaj** [1] **Habiba Sarhan** [1] **Morteza Shahrezaye** [1] **Kolja Hegelich** [1]
Christian Montag [2] **Ralf Schneider** [3] **Erik Cambria** [4]

Abstract

AI coding harnesses couple language models to shells, filesystems, and network sockets so that agents can resolve issues and run tests. That coupling turns the harness into an automated Living-off-the-Land (LotL) platform: once an adversary can inject instructions into retrieved content, dual-use binaries such as `bash`, `curl`, and `find` become the payload rather than a vulnerability to be separately exploited. We argue that security in agentic systems is not an emergent behavioral property of individual language models, but a structural property of the cognitive architecture and of the composition of permissions across interacting agents and humans. Many high-severity harness failures are classical OS and software problems—symlink races, path traversal, subprocess escape, SSRF, credential exposure, build-hook persistence—accelerated by agentic tool chaining. Static regex blacklists miss flag permutations and structural attacks, whereas LLM-as-a-Judge guardrails ask a second model to guess physical invariants at tens to hundreds of milliseconds per call while over-blocking benign work.

We present ActiveSecHarness, a neurosymbolic reference-monitor architecture organized around Separation of Intelligence and Authority: intelligence may propose actions, but it must never define its own authority. The design separates four pillars. *Authority* is carried by Macaroon-inspired HMAC capability tokens that attenuate monotonically under delegation across four Trust and Authority Rings. *Information flow* is governed by a product taint lattice $\mathcal{L} = \mathcal{L}_{\text{trust}} \times \mathcal{L}_{\text{sens}}$ (where $\mathcal{L}_{\text{trust}} = \{\mathsf{Trusted} \sqsubseteq \mathsf{Sanitized} \sqsubseteq \mathsf{Derived} \sqsubseteq \mathsf{Untrusted}\}$ and $\mathcal{L}_{\text{sens}} = \{\mathsf{Public} \sqsubseteq \mathsf{Sensitive}\}$) and a compositional Separation-of-Duties ledger over shared workspace provenance, so collaborating agents cannot launder secrets through disk. *Enforcement* mediates tool invocation, process capability, and OS/subprocess resources through deterministic guards. *Verification and applications* bind tools to extended bytecode certificates, seal split-horizon dual-key audit logs, and close a self-healing verification loop (`CodeVerifier`, `DualModelReviewer`, `SecurityTestSynthesizer`) paired with a progressive pedagogical `UserWikiManager` (`wiki_user/`) that builds cognitive immunity in human operators.

On N=252 fixtures (120 attack, 120 benign, 12 five-stage kill chains) we compare unguarded, static regex, live local LLM-judge (`qwen3:4b`), Ring 0, and hybrid (LLM advisory + Ring 0 authoritative) regimes. Attack block rates are 0.000, 0.439, 1.000, 1.000, and 1.000; false positive rates are 0.000, 0.000, 0.983, 0.000, and 0.000 (Clopper–Pearson 95% CIs). The live judge blocks attacks only by refusing 98.3% of benign SWE work at 142.3 ms median latency, whereas Ring 0—and the Ring 0-authoritative hybrid—reach ABR=1.000 at FPR=0.000 in 12.7 μs (11,200× faster). In a multi-step trajectory ablation (12 kill chains, 12 benign SWE workflows), even an oracle Conseca-style stateless task policy misses 4/12 intra-scope trajectories (ABR=0.667) and a coarse Boolean taint lock rejects 83.3% of benign issue-driven workflows (FPR=0.833), whereas the Ring-0 product lattice and `WorkspaceProvenanceLedger` achieve ABR=1.000 and FPR=0.000 at a

[1]Evercursion – GaiaLogic Unipessoal Lda. [2]University of Macau [3]Allianz Group [4]Nanyang Technological University, Singapore. Correspondence to: Simon Hegelich <simon.hegelich@evercursion.ai>.

Preprint. November 11, 2026.

constant $\approx 20\,\mu$s per step for $k{=}1\ldots50$.

1. Introduction

Coding agents are useful because they act. Systems such as SWE-agent couple a language model to an agent–computer interface that edits files, navigates repositories, and runs tests and other programs (Yang et al., 2024, p. 1). OpenHands goes further, equipping agents with "a docker-sandboxed operating system with a bash shell, a web browser, and IPython server" (Wang et al., 2025, p. 2). Cybersecurity agents make the same design choice in offensive form: Fang et al. give agents a terminal and a Python interpreter so that tools such as `curl` can be used against live targets (Fang et al., 2024, p. 3), and Cybench evaluates agents whose actions are bash commands executed in Kali Linux (Zhang et al., 2025, p. 3). In each case the harness deliberately reuses the host's dual-use binaries rather than inventing a closed virtual instruction set. That is the Living-off-the-Land (LotL) duality of the coding harness: the same architectural surface that enables software engineering and red-team research also turns the agent into an automated LotL executor once its instruction stream can be manipulated.

The manipulation channel is no longer speculative. Indirect prompt injection shows that "LLM-Integrated Applications blur the line between data and instructions," so that adversaries can "remotely (without a direct interface) exploit LLM-integrated applications by strategically injecting prompts into data likely to be retrieved" (Greshake et al., 2023, p. 1). Tool-integrated agents amplify the blast radius: InjecAgent documents how malicious instructions embedded in external content aim "to manipulate these agents into executing detrimental actions against users" (Zhan et al., 2024, p. 1), and AgentDojo notes that a successful injection "can allow an external attacker to take actions (and call tools) on behalf of the user" (Debenedetti et al., 2024, p. 1). ToolEmu's risk analysis makes the consequence concrete: even the safest evaluated language-model agent still exhibits severe tool failures "23.9% of the time according to [their] evaluator" (Ruan et al., 2024, p. 1).

What follows is often misdiagnosed as a novel model attack. Symlink and TOCTOU races, path traversal, subprocess escape, environment and `PATH` injection, SSRF and DNS rebinding, credential exposure, resource exhaustion, build-hook persistence, cross-agent exfiltration through shared files, and persistent workspace poisoning are classical OS and software problems. Agentic tool chaining accelerates them by composing dual-use binaries under ambient authority; it does not invent a new physics of compromise. The central thesis of this paper is therefore not merely that coding harnesses need better individual filters, but that **security in agentic systems is a structural property of the architecture and of the composition of permissions—not a behavioral property of individual agents**. In our companion mathematical theory of open-ended intelligence, we show that cognitive capability is not the consequence of model scale alone, "but of a particular organizational structure," so that "the question worth asking shifts from which model to which system" (Hegelich et al., 2026, pp. 1–2). Overcoming the failure modes of both blind statistical token predictors and brittle single-layer rules requires "designing a composite architecture that keeps these requirements structurally separate while allowing them to integrate" (Hegelich et al., 2026, p. 2). Crucially, when multiple bounded agents interact through shared external memory, "[t]he collective state is not the union of individual states but the individual states together with the relations coupling them" (Hegelich et al., 2026, p. 32). Applied to security, this theorem explains why aligning individual agents in isolation is structurally insufficient: two agents whose individual steps each appear benign can still compose across shared workspace state into an unauthorized exfiltration channel unless authority and information flow are enforced across their composition.

Two common industry responses miss that architectural point. Static string or regex blacklists can catch a handful of literal patterns, yet they fail on flag permutations, Base64 or Hex pipe-to-shell payloads, spoofed archive headers, magic-byte disguises, and Unicode bidirectional (Bidi) Trojan Source encodings that make source appear different to a compiler and to the human eye (Boucher & Anderson, 2023, p. 1). Delegating the same checks to a secondary LLM-as-a-Judge is attractive as a product feature and brittle as a protection mechanism. A judge must reason correctly about whether an action violates a rule, yet systematic evaluation of LLM logical reasoning across deductive, inductive, and abductive settings—using content-neutral problems that separate reasoning from text comprehension—traces model failures to both evidence selection and the reasoning process itself (Xu et al., 2025). Heuristic detectors "provide [no] guarantee of security and regularly fall short to new attacks in practice" (Debenedetti et al., 2025, p. 3). Saltzer and Schroeder's principle of complete mediation requires that "[e]very access to every object must be checked for authority" (Saltzer & Schroeder, 1975, p. 1280); economy of mechanism further insists that protection

designs stay "as simple and small as possible," because "design and implementation errors that result in unwanted access paths will not be noticed during normal use" (Saltzer & Schroeder, 1975, p. 1280). Asking a probabilistic model to decide decompression ratios, canonical path containment, SSRF IP resolution, or password entropy violates both principles and pays a latency tax four orders of magnitude above a deterministic kernel.

A second classical failure mode appears once harnesses and Model Context Protocol (MCP) servers execute with the user's ambient authority across the cyber kill chain—from reconnaissance and weaponization of repository artifacts through exploitation, privilege escalation, and persistence or exfiltration. Hardy's confused deputy "runs with authority stemming from two sources" and "has no way to keep them apart" (Hardy, 1988, p. 36). MCP deployments compound the problem: tool selection commonly relies "on textual tool names and descriptions without cryptographic verification" (Hou et al., 2025, p. 17), and a post-installation rug pull can alter "an apparently benign MCP server" into malicious code (Hou et al., 2025, p. 19). The protocol itself "does not include mechanisms for runtime isolation or privilege control" (Hou et al., 2025, p. 24). Recent operational incidents show that this ambient-authority failure mode is already producing severe national-security and financial consequences. In late 2026, a defense contractor connected a personal AI coding agent with administrative credentials to the Pentagon's Non-classified Internet Protocol Router Network (NIPRNet), where the agent ran unmediated for two months and exfiltrated login credentials, passwords, and an internal network map to a high-risk third-party website vulnerable to prompt injection (Elliott, 2026). In the October 2026 Binance heist ($\approx$ \$6.34 B impacted), attackers trojaned a developer's local `pyenv` binary after a fake recruitment coding test and then pivoted through an internal AI tool to compromise two additional wallet engineers, harvest a container deployment permission token, and reach the hot-wallet signing service to authorize 50 on-chain EIP-7702 delegations (Binance Security Team, 2026; Evans, 2026)—confirming that an internal AI tool with host execution rights functions as "a remote-administration system with a chat window" whose decisive vulnerability is "what an AI tool may do, for whom, and with whose approval" (Evans, 2026). System-level defenses such as IsolateGPT (Wu et al., 2025), AirGapAgent (Bagdasarian et al., 2024), Conseca (Tsai & Bagdasarian, 2025), and CaMeL (Debenedetti et al., 2025) show that execution isolation, contextual task scoping, and explicit control–data separation help; yet because stateless per-action policies cannot track how individually benign steps compose across multi-step trajectories (Tsai & Bagdasarian, 2025, p. 5), coding harnesses still need an OS-facing reference monitor and provenance ledger that mediate archives, sockets, credentials, dual-use binaries, and forensic audit trails without handing reconnaissance blueprints to an injected workspace agent.

Equally misplaced is the prevailing model-centric cybersecurity discourse. Public alarm frequently centers on the claim that frontier language models have grown so intrinsically dangerous that they autonomously hacked enterprise targets; the policy reflex is to lobotomize model weights via refusal tuning or to restrict access to open-weight security and code models on hosting platforms such as HuggingFace. Because "[o]pen-ended intelligence is a property of a recursively extended cognitive process, not of an individual physical substrate" (Hegelich et al., 2026, p. 37), attributing system compromise to "dangerous model weights" mistakes an execution-boundary failure for a weight-level property. Where an autonomous agent compromises a target or a host environment, the decisive failure is architectural: the harness or target lacked complete mediation, least privilege, and compositional permission enforcement. Conversely, crippling the model's security reasoning at the weight level creates a dangerous dual-use defense paradox: defenders urgently need uncrippled, high-capability models inside a secure harness to audit complex codebases, synthesize adversarial regression tests, and repair vulnerabilities. And because the human operator—particularly the "vibe coder" who prompts language models to assemble applications without formal security training under schedule pressure, generating code at a volume that overwhelms manual review while logic-level backdoors fly under the radar of traditional static scanners (SAST) (Elliott, 2026)—is an active participant in the coupled cognitive system (Hegelich et al., 2026, p. 37) and a prime target of social engineering, compositional security fails if the human repeatedly rubber-stamps dangerous prompts or waives guards without understanding why. A secure harness must therefore verify generated code, heal vulnerabilities before execution, and systematically educate the operator over time.

Framed in cognitive-architectural terms, a secure coding harness is a neurosymbolic AI system by definition. As Cambria and collaborators observe, "[s]ubsymbolic techniques do not require much time nor effort to perform classification but they are data-dependent and function in a black-box manner" (Cambria et al., 2020, p. 107), and "even the finest AI models are vulnerable to adversarial examples" (Cambria et al., 2022,

p. 3829). The latest iteration of this line, SenticNet 9, carries the argument into the LLM era: because LLMs remain limited by hallucination and opaque reasoning, it uses generative models only to discover candidate conceptual primitives and keeps the resulting reasoning in an explicit symbolic layer, outperforming GPT-4.1 on three affective computing tasks while leaving every decision inspectable (Cambria et al., 2026). Even when large language models use natural language as a "shared interface between neural pattern-matching and symbolic control" (Mao et al., 2026, p. 1), a purely language-mediated pipeline means that "the model is not enforcing symbolic well-formedness in the technical sense" (Mao et al., 2026, p. 2). Consequently, "even if real parrots or stochastic ones (LLMs) produce appropriate linguistic responses to task-related φ_i prompts, we would not deem their linguistic behavior trustworthy" on their own, which is why "humans have to watch over them and correct them in mission-critical tasks" (Cambria et al., 2023, p. 67). ActiveSecHarness operationalizes neurosymbolic AI at the operating-system boundary to "mitigate unexpected model behavior and inject interpretability by combining learnable parameters (neuro) with predefined knowledge bases or templates (symbolic)" (Xing et al., 2023, p. 1712): the subsymbolic neural model proposes plans, code, and security reviews in Rings 2–3, while Ring 0 enforces formal symbolic capability algebra, information-flow labels, AST well-formedness, and cryptographic state transitions. The governing slogan is deliberate: intelligence may propose actions, but it must never define its own authority.

We therefore treat the coding harness as a structural and compositional protection problem and contribute four interlocking pieces:

1. A six-domain LotL threat taxonomy (Domains A–F) mapped onto STRIDE, MITRE ATT&CK, and the cyber kill chain, together with a classical-primitives framing that states four mediation requirements—complete mediation, resource containment, information-flow control, and temporal integrity (Section 2).

2. ActiveSecHarness, structured around (i) Authority: four Trust and Authority Rings with Macaroon-inspired HMAC capability chains; (ii) Information Flow: a product taint lattice $\mathcal{L} = \mathcal{L}_{\text{trust}} \times \mathcal{L}_{\text{sens}}$ and compositional Separation of Duties over workspace provenance; (iii) Enforcement: three mediation boundaries mapped to concrete OS resource interceptors; and (iv) Verification, Self-Healing, and Pedagogical Defense: extended tool certificates, split-horizon audit logs, a closed-loop `CodeVerifier` / `DualModelReviewer` / `SecurityTestSynthesizer` repair pipeline, and the progressive `UserWikiManager` security academy (Section 3, Section 4).

3. Three formal guarantees in the appendix: Separation of Intelligence and Authority via monotonic attenuation; monotonic information-flow security under the product lattice (distinguishing sanitization from redaction while preserving isolated SWE tests under Derived); and compositional multi-agent non-exfiltration across shared workspace state (Section A).

4. An empirical evaluation on N=252 fixtures (120 attack, 120 benign, 12 kill chains) across five regimes—unguarded, regex, live local LLM-judge, Ring 0, and hybrid—plus a multi-step trajectory ablation comparing an idealized oracle Conseca-style stateless task policy, a coarse Boolean taint lock, and Ring 0's product lattice + `WorkspaceProvenanceLedger` across 12 attack kill chains, 12 benign SWE trajectories, and trajectory lengths k=1 . . . 50 (Section 5).

2. Classical Threat Taxonomy for Coding Harnesses

A coding harness is not a chat interface with a few plugins. It is a process that can open shells, rewrite trees, and dial sockets under the operator's identity. Saltzer and Schroeder already warned that "[e]very program and every user of the system should operate using the least set of privileges necessary to complete the job" (Saltzer & Schroeder, 1975, p. 1281), and that access decisions should rest on "permission rather than exclusion," so that "the default situation is lack of access" (Saltzer & Schroeder, 1975, p. 1280). Coding agents routinely invert that default: once installed, they inherit ambient filesystem, credential, and network authority, then accept untrusted repository text, issue trackers, and tool descriptions as if those inputs were inert data. We organize the LotL surface into six domains and a concrete sub-vector catalog (Appendix A), each annotated with STRIDE categories and representative MITRE ATT&CK techniques. The goal is not encyclopedic coverage of every CVE; it is a mediation checklist that a Ring 0 kernel can implement without asking a language model whether a ZipBomb "looks dangerous."

2.1. Classical Attack Primitives under Agentic Acceleration

Before the domain catalog, we state a thesis that shapes the rest of the paper. Many high-severity harness failures are not exotic model behaviors; they are classical attack primitives that agentic tool chaining merely accelerates. Symlink and TOCTOU races rewrite destinations between policy check and open. Path traversal escapes workspace roots through `../` and link following. Subprocess escape turns an allowlisted binary into a shell via flags such as `find -exec`. Environment and `PATH` injection smuggle credentials and substitute trojaned executables into child processes. SSRF and DNS rebinding reach loopback, RFC1918, and cloud metadata after a hostname has already passed a textual allowlist. Credential exposure leaks keys through files, logs, and tool arguments. Resource exhaustion (fork, memory, archive bombs) denies the host. Build hooks and SCM triggers (`.git/hooks`) persist across sessions. Cross-agent exfiltration uses a shared workspace file as a covert channel between differently scoped agents. Persistent workspace poisoning plants instructions or artifacts that later sessions will trust.

We do not claim that every existing coding harness leaves all of these classes unprotected. We claim that the classes sit below the tool-call abstraction that product UIs and MCP schemas expose, and that they must therefore be enforced at the resource boundary—where pathnames are resolved, processes are spawned, and sockets are connected. Four requirements follow: *Complete Mediation* (every access is checked (Saltzer & Schroeder, 1975, p. 1280)), *Resource Containment* (quotas, namespaces, and scrubbed environments bound blast radius), *Information-Flow Control* (labels travel with data across tools and agents), and *Temporal Integrity* (check and use cannot be raced apart). Table 1 maps each primitive to the primary requirement that defeats it; Domains A–F below are the LotL-facing organization of the same surface.

2.2. Ambient Authority and the Confused Deputy

Hardy's classic observation still fits the harness: the privileged component "serves two masters and carries some authority from each," yet "has no way to keep them apart" (Hardy, 1988, p. 36). In the modern setting one master is the human operator who installed the agent; the other is whichever remote artifact or MCP tool description the agent last retrieved. Capabilities, in Hardy's sense, unify designation and authorization so that a callee "would merely refer to a capability provided by the invoker" (Hardy, 1988, p. 37). Without that discipline, GTFOBins-style shell escapes, environment-variable credential inheritance, and network egress all execute under a single ambient principal. Separation of privilege—"a protection mechanism that requires two keys to unlock it" (Saltzer & Schroeder, 1975, p. 1281)—is precisely what ambient YOLO modes discard when a single session may read secrets, run shells, and open sockets.

Table 1. Classical primitives accelerated by agentic tool chaining, and the primary harness requirement that addresses each.

Primitive	Primary requirement
Symlink / TOCTOU	Temporal integrity + complete mediation
Path traversal	Complete mediation (canonical paths)
Subprocess escape	Resource containment (flag / binary policy)
Env / `PATH` injection	Resource containment (scrubbed env)
SSRF / DNS rebinding	Temporal integrity (pre-connect IP pin)
Credential exposure	IFC + DLP mediation
Resource exhaustion	Resource containment (quotas)
Build hooks / SCM triggers	Complete mediation (sensitive paths)
Cross-agent exfiltration	Compositional IFC / SoD
Workspace poisoning	IFC (taint) + temporal integrity

2.3. Domain A: LotL Binary Abuse and Self-Tampering

Domain A covers misuse of pre-installed dual-use binaries and of the harness's own security configuration. Sub-vector A.1 is classical living-off-the-land execution (ATT&CK T1059): `find -exec`, `curl | sh`, `python -c`, and similar compositions that never drop a custom implant. A.2 covers environment-variable credential inheritance (T1552.001), where child processes receive `AWS_*`, `OPENAI_API_KEY`, or shell history simply because the harness scrubbed nothing. A.3 is Ring 0 self-tampering (T1562.001): an agent that can rewrite its own policy files or unload guards has already escaped mediation. A.4 covers persistence through `.git/hooks` and related SCM triggers (T1546). Cyber agents that "output a response that includes an action (a bash command . . .) which is then executed in the environment" (Zhang et al., 2025, p. 3) make Domain A the default rather than an edge case. STRIDE mappings concentrate on Elevation of Privilege and Tampering.

2.4. Domain B: Artifact and Archive Vectors

Domain B treats files as hostile programs. B.1 is the spoofed-header ZipBomb (T1499): declared sizes lie,

nested archives explode, and any guard that trusts ZIP headers fails closed only by accident. B.2 is Zip-Slip path and symlink traversal (T1083), writing outside the destination tree. B.3 covers polyglot and magic-byte disguises (T1036), for example an ELF image renamed to `.txt`. B.4 is Trojan Source: source "maliciously encoded so that it appears different to a compiler and to the human eye," exploiting Unicode so that "tokens are logically encoded in a different order from the one in which they are displayed" (Boucher & Anderson, 2023, p. 1). Because Bidi controls inside comments and strings are accepted by most compilers (Boucher & Anderson, 2023, p. 3), an operator approving a "sudo" prompt can ratify a command that is not the one displayed. STRIDE emphasis is Tampering and Denial of Service.

2.5. Domain C: Network, SSRF, DNS Rebinding, and Supply Chain

Domain C begins where the harness leaves the workstation. C.1 is cleartext HTTP downgrade. C.2 is SSRF to loopback, RFC1918 space, or cloud metadata endpoints such as `169.254.169.254` (T1071). C.3 is DNS-rebinding time-of-check/time-of-use: a name resolves to a public address for the policy check and to a private address for the socket. C.4 covers unverified external model or tool endpoints that silently exfiltrate context. Fail-safe defaults require that such targets be explicitly permitted rather than silently reachable (Saltzer & Schroeder, 1975, p. 1280). STRIDE maps primarily to Information Disclosure and Spoofing.

2.6. Domain D: Credentials, Dictionary Mutations, and Secret DLP

Domain D covers secrets in both directions. D.1 is weak and mangled dictionary passwords. Wheeler shows that LUDS character-class rules remain "a conclusively burdensome and ineffective security practice" (Wheeler, 2016, p. 157), while leetspeak demangling recovers dictionary words from mangled inputs (Wheeler's running example demangles to `abalone`) (Wheeler, 2016, p. 163). D.2 is bidirectional secret DLP (T1552): cleartext API keys, PEM material, and JWTs must be blocked on writes *and* redacted or refused on reads that would feed exfiltration tools. STRIDE focus is Information Disclosure and Elevation of Privilege.

2.7. Domain E: Sandbox Escape and Resource Starvation

Domain E is confinement failure. Even when coding harnesses wrap agent sessions in OS-level containers, "containers virtualize resources at the OS level where all the containers share the same OS" (Zhong et al., 2022, p. 2), making explicit per-container resource allocation and limits foundational for "resource isolation control between containers" (Zhong et al., 2022, p. 7). Without per-call mediation inside the container boundary, an injected agent still enjoys ambient access to everything mounted into that container. E.1 is host-path escape outside the workspace (`/etc/passwd`, `~/.ssh`). E.2 is fork, memory, or disk bombs (T1499). E.3 is unapproved binary flags that convert an allowlisted tool into a shell (`find -exec`, `tar --to-command`, `git -c core.fsmonitor`). Psychological acceptability matters here: if the human interface is not "designed for ease of use, so that users routinely and automatically apply the protection mechanisms correctly" (Saltzer & Schroeder, 1975, p. 1281), operators will click through opaque approval dialogs. STRIDE maps to Elevation of Privilege and Denial of Service.

2.8. Domain F: Cognitive and Multi-Agent Vectors

Domain F is where language-model control flow meets classical authority. F.1 is indirect prompt-injection taint escalation: retrieved content that should be treated as data becomes instructions (Greshake et al., 2023, p. 1). F.2 is confused-deputy sub-agent delegation and social-engineering "sudo" traps, including MCP rug-pull mutation of tool implementations (Hou et al., 2025, p. 19). F.3 is the Lethal Trifecta: combining sensitive read, shell execution, and network egress inside one session—or across collaborating agents that share workspace files—so that stolen material can leave the host. These vectors do not replace Domains A–E; they orchestrate them. A taxonomy that stopped at string matching would systematically undercount the attack that never looks like a banned token.

2.9. Kill-Chain Acceleration and the Forensic Reconnaissance Paradox

Domains A–F do not occur as isolated one-shot events; in an unguarded or naively guarded coding harness, they compose to accelerate every stage of the classical cyber kill chain:

1. **Reconnaissance:** An injected agent enumerates workspace trees, environment variables (A.2), SSH configs (E.1), and—critically—the harness's own telemetry and audit logs to map internal paths, prior failure modes, and redacted or cleartext secrets.

2. **Weaponization and Delivery:** Malicious payloads arrive disguised as benign repository artifacts—spoofed-header ZipBombs (B.1), Zip-Slip archives (B.2), magic-byte polyglots (B.3), Trojan Source Bidi comments (B.4) (Boucher & Anderson, 2023, p. 3), or poisoned issue text and MCP servers (F.1, F.2) (Hou et al., 2025, p. 19)—or are generated inline when a small model writes vulnerable application code for an unsuspecting "vibe coder."

3. **Exploitation:** Instead of corrupting memory, the attacker exploits the agent's instruction/data confusion (Debenedetti et al., 2024, p. 1) or tricks the human operator into rubber-stamping an obfuscated Base64 pipe-to-shell prompt.

4. **Privilege Escalation and Lateral Movement:** The compromised agent abuses GTFOBins flags (A.1, E.3), confused-deputy sub-agent delegation (F.2) (Hardy, 1988, p. 36), Ring 0 self-tampering (A.3), or SSRF and DNS rebinding against internal metadata services (C.2, C.3).

5. **Persistence and Exfiltration:** The attacker plants `.git/hooks` triggers (A.4) and closes the Lethal Trifecta (F.3) by combining sensitive file reads (D.2) with network egress (C.1, C.4), including laundering through a second agent's workspace read.

Mapping the kill chain exposes a fundamental tension in harness logging. Defenders require rich execution and violation telemetry for *ex-post* forensic reconstruction; yet if those logs sit in readable workspace files or single-tier JSONL streams, an adversary who achieves Stage 1 reconnaissance inside the workspace can read the audit trail as a complete blueprint of system paths, guard rules, and credential locations. As we show in Section 3, Section 4, resolving this paradox requires applying Saltzer and Schroeder's separation of privilege (Saltzer & Schroeder, 1975, p. 1281) directly to the audit log via split-horizon dual-key encryption, and treating the human operator not as an infallible oracle but as a participant who must be protected by automated code verification and continuous security education.

3. The ActiveSecHarness Architecture

ActiveSecHarness adapts classical protection rings to agentic coding harnesses as a composite neurosymbolic architecture. As established in our mathematical foundation for open-ended intelligence, cognitive capability is not a product of scale alone, "but of a particular organizational structure" (Hegelich et al., 2026, p. 2) whose "[f]irst law is a strict duality: the system needs an object-level structure and a distinct meta-level structure, kept separate. The object level is responsible for interacting with the world. It conducts observations, performs operations, generates hypotheses, and stores empirical knowledge. The meta level acts on the object level itself rather than on the world directly" (Hegelich et al., 2026, p. 2). Attempting to collapse this duality by asking a single language model—or a second language model acting as an inline judge—to enforce its own physical boundaries confuses object-level hypothesis generation with meta-level authority.

Instead, ActiveSecHarness "keeps these requirements structurally separate while allowing them to integrate" (Hegelich et al., 2026, p. 2). As Cambria et al. (2023, p. 66) argue, "[o]ld-school (symbolic) AI was better at Symbol Grounding but it was not scalable nor flexible. New deep-learning-based (subsymbolic) AI, instead, is very scalable and flexible but it does not handle symbols. The best of both worlds could be achieved through a hybrid (neurosymbolic) AI that leverages the strengths of both symbolic and subsymbolic models to overcome their respective limitations." In ActiveSecHarness, "symbolic representations and reasoning are used to provide high-level context and logic, while neural networks are employed for pattern recognition, learning from data, and capturing low-level features" (Xing et al., 2023, p. 1712), directly addressing what Cambria et al. (2022, p. 3829) identify as the five core limitations of purely subsymbolic systems: "dependency, reproducibility, trustworthiness, interpretability, and explainability." Ring 0 mediation and the self-verifying ledger make security enforcement "reproducible, because each reasoning step can be explicitly recorded and replicated through each iteration" (Cambria et al., 2022, p. 3829). SenticNet 9 names the deeper obstacle as the symbol grounding problem: representations that are not tied to real-world meaning leave a system's decisions opaque and hard to interpret (Cambria et al., 2026). Ring 0 is grounded in exactly this sense. Its decisions refer to resolved canonical paths, pinned socket addresses, measured decompression ratios, and signed capability scopes, never to the natural-language text that describes an action, so an injected instruction that rephrases a dangerous call does not change what Ring 0 checks.

The architecture is organized around Separation of Intelligence and Authority under four pillars—Authority, Information Flow, Enforcement, and Verification—coupled to a closed-loop Verification and Pedagogical

Defense layer.

3.1. Authority: Trust and Authority Rings and Capability Chains

Definition 3.1 (Trust and Authority Rings)**.** We use *Trust and Authority Rings*, not hardware CPU rings. Ring 0 is the immutable security kernel: a non-bypassable reference monitor that mediates every tool call before and after execution and that cannot be rewritten by any agent token. Ring 1 is the human-sovereign layer: context-isolated command deobfuscation, explain-before-sudo ceremonies, and the progressive `UserWikiManager` curriculum that protect the operator against approval fatigue, Base64/Hex pipe-to-shell payloads, and Trojan Source Unicode (Boucher & Anderson, 2023, p. 1). Ring 2 hosts privileged orchestrators, independent security reviewers, and scoped "sudo" agents that operate only under short-lived, cryptographically attenuated capability grants (for example, laboratory pentesting inside `10.10.0.0/16`). Ring 3 hosts unprivileged workers: sandboxed, offline-by-default sub-agents restricted to canonical workspace paths and strict tool subsets.

Least privilege (Saltzer & Schroeder, 1975, p. 1281) and complete mediation (Saltzer & Schroeder, 1975, p. 1280) are structural rather than advisory: a Ring 3 worker never receives ambient authority, and Ring 0 checks every access even when Ring 1 has already shown a human a cleaned command string. In organizational terms, the four rings instantiate the institutional checks and balances that human high-reliability organizations use to govern fallible actors: rather than expecting a single generative agent to police itself, the architecture separates *generative* problem-solving in Rings 2–3 from *bureaucratic* procedural verification (`DualModelReviewer`, `ToolCertificate`) and a deliberately restrictive, need-to-know *pathological* gatekeeper in Ring 0 empowered to halt execution and revoke authority immediately upon tripwire contact (Westrum, 2004; Yu, 2024, p. ii22). IsolateGPT observes that LLM-app ecosystems "resemble the settings of earlier computing platforms, where there was insufficient isolation between apps and the system" (Wu et al., 2025, p. 1); our rings push isolation down to OS-facing tool mediation rather than stopping at app-level hub-and-spoke routing.

Ambient authority is replaced by HMAC-SHA256 chained `CapabilityToken` objects in the spirit of macaroons, which "embed caveats that attenuate and contextually confine when, where, by who, and for what purpose a target service should authorize requests" (Birgisson et al., 2014, p. 1). Integrity follows the nested construction in which "each such value itself is used as the HMAC key for the next message" (Birgisson et al., 2014, p. 4). Tokens additionally bind parent-subject identity, per-call nonces, and remaining quotas so that replay and quota inflation fail closed.

Definition 3.2 (Monotonic attenuation)**.** Let τ be a capability token with scope $\mathrm{Scope}(\tau)$ over allowed tools, canonical path prefixes, binary allowlists, network CIDR or domain targets, time-to-live, and call quotas. A child token τ_{child} minted from τ_{parent} is valid only if

$$\mathrm{Scope}(\tau_{\mathrm{child}}) \subseteq \mathrm{Scope}(\tau_{\mathrm{parent}}).$$

Delegation may add caveats; it may never widen them.

This is Hardy's capability lesson restated for multi-agent harnesses: designation and authorization travel together, so a sub-agent cannot inherit the parent's ambient rights merely by being spawned (Hardy, 1988, p. 37). CaMeL similarly "uses a notion of a capability to prevent the exfiltration of private data over unauthorized data flows by enforcing security policies when tools are called" (Debenedetti et al., 2025, p. 1); we keep that discipline while extending the protected object set to LotL binaries, archives, sockets, and shared workspace labels.

3.2. Information Flow: Product Lattice and Compositional SoD

Indirect prompt injection succeeds when retrieved text is treated as authority (Greshake et al., 2023, p. 1). AirGapAgent restricts accessible data to what a task needs (Bagdasarian et al., 2024, p. 1); we replace a coarse Boolean lock with a two-dimensional product lattice that separates provenance integrity from confidentiality while preserving ordinary software-engineering utility after reading an untrusted issue.

Definition 3.3 (Product information-flow lattice)**.** Let provenance trust be ordered by the four-element join-semilattice $(\mathcal{L}_{\mathrm{trust}}, \sqcup)$ with

$$\mathsf{Trusted} \sqsubseteq \mathsf{Sanitized} \sqsubseteq \mathsf{Derived} \sqsubseteq \mathsf{Untrusted},$$

and data confidentiality by the two-element lattice $\mathcal{L}_{\mathrm{sens}} = (\{\mathsf{Public} \sqsubseteq \mathsf{Sensitive}\}, \vee)$. Each session and workspace artifact carries a composite label $\ell = (\tau, \sigma)$ in the product lattice $\mathcal{L} = \mathcal{L}_{\mathrm{trust}} \times \mathcal{L}_{\mathrm{sens}}$ with componentwise join $(\tau_1, \sigma_1) \sqcup (\tau_2, \sigma_2) = (\tau_1 \sqcup \tau_2, \sigma_1 \vee \sigma_2)$. Ingesting external content raises τ; reading secret-bearing sources raises σ to $\mathsf{Sensitive}$. When $\sigma = \mathsf{Public}$, action classes are gated by τ: $\mathsf{Derived}$ permits workspace read/write, archive extraction, and isolated tests (`SHELL_TEST_ISOLATED`, e.g. `pytest` without network) while refusing $\mathsf{SHELL_ARBITRARY}$ and

NETWORK_EGRESS; Untrusted permits local inspection only. Whenever $\sigma = \mathsf{Sensitive}$, arbitrary shell and network egress are blocked regardless of τ. Two distinct downward operations are mediated by Rings 0–1: *Sanitization* (structural representation cleaning, such as stripping Trojan Source Bidi controls or deobfuscating syntax) maps $\tau \mapsto \mathsf{Sanitized}$ while leaving σ unchanged; *Redaction / Authorized Declassification* (`CredentialGuard` HMAC scrubbing or an explicit Ring 1 operator waiver) is the only transition that clears $\sigma \mapsto \mathsf{Public}$.

Separating $\mathcal{L}_{\text{trust}}$ from $\mathcal{L}_{\text{sens}}$ resolves two conflations that plague ad-hoc taint flags. First, reading a GitHub issue raises provenance to (Derived, Public), allowing local unit tests to run inside `SandboxGuard` while keeping arbitrary shells and network egress closed. Second, structural sanitization of an untrusted string never accidentally declassifies a secret: cleaning Unicode or shell syntax lowers τ to Sanitized but preserves $\sigma = \mathsf{Sensitive}$ until secret material is deterministically redacted.

Saltzer and Schroeder prefer mechanisms that need "two keys to unlock" (Saltzer & Schroeder, 1975, p. 1281). Intra-session toxic combinations are necessary but not sufficient once agents share a workspace. As shown in our formal theory of collective cognitive systems, when bounded agents interact through persistent external memory, "[t]he collective state is not the union of individual states but the individual states together with the relations coupling them" (Hegelich et al., 2026, p. 32). A security policy that inspects each agent's state in isolation is therefore blind to vulnerabilities that emerge purely from the coupling relation across shared files.

Definition 3.4 (Compositional Separation of Duties)**.** Ring 0 maintains a `WorkspaceProvenanceLedger` mapping each canonical path f to a composite label $\ell(f) = (\tau_f, \sigma_f) \in \mathcal{L}$ and origin side-effect set $\Sigma(f)$. When agent A writes f, $\ell(f) \leftarrow \ell(f) \sqcup \ell(A)$ and $\Sigma(f)$ absorbs A's active effects. When agent B reads f, $\ell(B) \leftarrow \ell(B) \sqcup \ell(f)$ and $\Sigma_B \leftarrow \Sigma_B \cup \Sigma(f)$. A call is rejected if the resulting state would close a toxic combination (combining $\sigma = \mathsf{Sensitive}$ or READ_SENSITIVE with NETWORK_EGRESS or EXEC_PROCESS) unless f has undergone authorized redaction/declassification to (Sanitized, Public). A polynomial greedy partitioner splits single-agent plans across attenuated child tokens when needed.

Hence Agent A cannot read a secret, write it to disk, and let Agent B exfiltrate it: the ledger mediates the coupling relation across external workspace memory (Hegelich et al., 2026, p. 37). The Lethal Trifecta becomes a deterministic check over composed provenance rather than a prompt to be argued with. Moreover, pairing task-scoped capability caveats with `WorkspaceProvenanceLedger` directly resolves the *agent trajectory problem* left open by just-in-time policy generators such as Conseca (Tsai & Bagdasarian, 2025, p. 5). Conseca uses an isolated model over trusted context to generate per-task policies—a mechanism that maps naturally to proposing initial Ring 2 `CapabilityCaveat` scopes—yet its policies "currently check individual actions, regardless of what actions came prior," leaving open how to "protect against seemingly harmless single actions composing in inappropriate ways" because static or generated rules cannot enumerate "multiple trajectories of potentially infinite length" (Tsai & Bagdasarian, 2025, p. 5). Instead of enumerating execution paths a priori, Ring 0 folds any single- or multi-agent trajectory of arbitrary length into the monotone product state $(\ell(A), \Sigma_A)$ and workspace provenance map $\ell(f) = (\tau_f, \sigma_f)$ in $\mathcal{O}(1)$ time per step, deterministically refusing the exact transition that would close a toxic trajectory.

3.3. Enforcement: Three Mediation Boundaries

Complete mediation (Saltzer & Schroeder, 1975, p. 1280) operates across three nested boundaries corresponding to the classical primitives of Section 2: (1) *Agent / tool-invocation mediation* (HMAC chain verification, nonce and quota checks, tool-certificate recomputation, product-lattice permission, and compositional SoD checks before arguments are interpreted); (2) *Process and capability mediation* (scrubbed environments, per-binary disallowed flags, call-class binding between `SHELL_TEST_ISOLATED` and `SHELL_ARBITRARY`, and attenuated CIDR scopes); and (3) *OS / subprocess resource mediation* (canonical path confinement, `RLIMIT_*` quotas, `O_NOFOLLOW` staging extraction, socket-level IP pinning, and canary-triggered lineage revocation). Table 2 maps the six host resource classes to their exact Ring 0 enforcement points; domain-specific guards are detailed in Section 4.

3.4. Verification and Self-Healing Code Pipeline

Because MCP tool selection often lacks cryptographic verification (Hou et al., 2025, p. 17) and rug pulls can rewrite a server after installation (Hou et al., 2025, p. 19), Ring 0 maintains a `CertifiedToolRegistry`.

Definition 3.5 (Tool certificate)**.** At registration, each tool is bound to an immutable `ToolCertificate` recording the SHA-256 digest of its Python bytecode

Table 2. Ring-0 enforcement points across host resource channels.

Resource	Ring-0 Interceptor	Enforced Invariant
Filesystem I/O	`SandboxGuard`, `CanaryGuard`	Canonical `realpath` lock; `O_NOFOLLOW`; canary lineage revocation
Archives	`ArchiveGuard`	64 KiB chunked quotas; staging `realpath` check; symlink/FIFO reject
Subprocesses	`SecurityKernel`, `SandboxGuard`	Binary allowlist; GTFOBins flag block; `RLIMIT_AS/CPU/NPROC/FSIZE`
Network Sockets	`NetworkGuard`	TLS enforcement; resolve-once DNS; RFC1918/metadata block; IP pin
Credentials	`CredentialGuard`	Reverse-cracker demangling; ≥ 4.0 bits/char entropy; bidirectional DLP
Shared State	`WorkspaceProvenance Ledger`	Product-lattice (τ, σ) join; cross-agent Lethal Trifecta refusal

(`__code__.co_code`), a normalized AST digest of its source, a dependency digest, an import-path lock, an environment allowlist, declared side-effect categories, and a publisher HMAC signature. On every invocation Ring 0 recomputes the digests and aborts on mismatch before arguments are interpreted.

Certification makes silent tool mutation detectable, while runtime privilege control remains inside Ring_0(Hou et al., 2025, p. 24). Yet mediating tool calls alone is insufficient if an agent writes vulnerable application code into the workspace and executes it. ActiveSecHarness closes that gap through a three-stage verification and self-healing loop:

1. **Ring** 0 `CodeVerifier`**.** Every generated or modified program is checked before execution against a three-tier scoping model (Section 4) that enforces universal AST invariants and executes dynamic adversarial probes inside an isolated SoD `test-runner` with zero network access.

2. **Ring** 2 `DualModelReviewer`**.** Deterministic Stage-A verification is paired with an independent Stage-B reviewer model in a clean context window. Either stage can set approved_for_execution = False, recommend model escalation when a small local coder repeatedly fails, and drive an automated Fix–Test–Verify repair loop.

3. **Ring** 2 `SecurityTestSynthesizer`**.** Novel semantic flaws spawn task-scoped probes admitted only under the Red–Green–Benign quartet: AST-safe, Red on the vulnerable draft, Benign-safe on normal inputs, and Green after repair.

3.5. Pedagogical User Defense and Cognitive Immunity (`UserWikiManager`)

In a compositional architecture, the human operator is not an external oracle standing outside the system; the operator is a coupled cognitive agent holding Ring 1 authority to mint capabilities and grant `ContextWaiver` overrides (Hegelich et al., 2026, p. 37). Under "vibe coding"—where developers and domain scientists prompt language models to assemble applications whose security invariants they cannot independently audit—the human becomes the primary target of social engineering and approval fatigue. If the harness merely blocks unsafe actions with opaque error codes, operators quickly learn to bypass guards or rubber-stamp dangerous prompts, collapsing the composition of permissions at Ring 1. Saltzer and Schroeder's principle of psychological acceptability requires that "the human interface be designed for ease of use, so that users routinely and automatically apply the protection mechanisms correctly" (Saltzer & Schroeder, 1975, p. 1281).

Ring 1's `UserWikiManager` operationalizes psychological acceptability as **pedagogical cognitive immunity**. Whenever `CodeVerifier`, `DualModelReviewer`, or `CommandDeobfuscator` intercepts a vulnerability or deobfuscates a trap, `UserWikiManager` records an immutable `UserLearningCase` and updates a local, human-centric security academy (`wiki_user/`, accessible interactively via `/learn` in the chat REPL). Before any lesson is written, `UserWikiManager` enforces canonical path confinement inside `wiki_user/`, rejects symlinks, strips Trojan Source Unicode (Boucher & Anderson, 2023, p. 1), and passes all snippets through `CredentialGuard.redact_secrets()`. Rather than overwhelming novices with raw CVE jargon, the curriculum automatically adapts explanation depth to the operator's cumulative resolved cases:

- **Level 1 (Cases 1–2, Foundations & Everyday Analogies):** Explains the flaw through concrete physical analogies, plain-language mechanics, and the specific social-engineering lures attackers use to trick developers and small coding models.

- **Level 2 (Cases 3–4, Secure Engineering Patterns):** Unlocks side-by-side diffs of the blocked unsafe draft versus the sandbox-verified repair, highlighting exact AST invariants, canonical path locks, and Separation-of-Duties boundaries.

- **Level 3 (Cases 5+, Adversarial & Cryptographic Mastery):** Unlocks deep exploit

mechanics—such as non-recursive overlapping deflate streams, TOCTOU symlink race windows, and GTFOBins flag escapes—alongside kernel and cryptographic countermeasures.

Table 3 illustrates two concrete lessons generated by `UserWikiManager` during live harness workflows, showing how automated Ring 0/2 interception and repair are translated into progressive human mastery.

4. Deterministic Ring-0 Interceptors and Self-Verifying Ledger

Ring 0 is only as strong as the checks it actually runs. The interceptors below are deliberately boring: streaming limits, canonical path tests, magic-byte tables, dictionary demangling, DNS resolution before connect, resource quotas, and honeypot canaries. None of them ask a language model whether an archive "seems" safe. Fail-safe defaults (Saltzer & Schroeder, 1975, p. 1280) mean that an unrecognized shape is denied.

4.1. ArchiveGuard

`ArchiveGuard` defends Domain B decompression and Zip-Slip paths. Untrusted ZIP header `file_size` fields are ignored. Decompression streams in 64 KiB chunks into an isolated staging directory under hard ceilings on per-entry bytes, total bytes, file count, and compression ratio. Member files are opened with `O_NOFOLLOW`; symlinks, FIFOs, and device nodes are rejected. After extraction, every staged path is re-checked with realpath containment against the staging root, then promoted atomically into the destination tree only if

$$\text{Path.resolve}(p).\text{is_relative_to}(\text{dest})$$

holds. Spoofed-header ZipBombs are therefore a resource-accounting and temporal-integrity problem rather than a content-classification problem: the TOCTOU window between write and promotion cannot introduce a link that escapes the staged root.

4.2. MalwareGuard

`MalwareGuard` inspects leading magic bytes for ELF, PE/MZ, Mach-O, and shebang signatures even when the filename claims `.txt`, `.jpg`, or `.md`. The check is a *format and executable-class* gate, not a claim of malware family attribution; optional YARA or ClamAV hooks can be attached without changing the mediation point. For source and configuration artifacts the guard strips and rejects Trojan Source Unicode Bidi controls (`U+202A`–`U+202E`, `U+2066`–`U+2069`) and selected zero-width homoglyphs, following Boucher and Anderson's demonstration that Bidi smuggling inside comments and strings survives ordinary compilation (Boucher & Anderson, 2023, p. 3). The invariant is display/execution agreement for any string that Ring 1 will show a human or that Ring 0 will pass to a shell.

4.3. CredentialGuard

`CredentialGuard` rejects weak secrets before they are written and redacts or blocks cleartext exfiltration on reads. Password checking inverts leetspeak substitutions (at-sign$_{\to}$`a`, `0`$_{\to}$`o`, dollar$_{\to}$`s`, `1`$_{\to}$`i/l`, `3`$_{\to}$`e`), strips common prefixes and suffixes, and matches against banned and project-context dictionaries, following the demangling strategy exemplified by zxcvbn (Wheeler, 2016, p. 163). We additionally require Shannon entropy of at least 4.0 bits per character and refuse LUDS-only policies that Wheeler showed to be ineffective (Wheeler, 2016, p. 157). Bidirectional DLP patterns cover cloud access keys, PEM blocks, and JWTs on both filesystem writes and reads that would feed Domain C egress, feeding the information-flow ledger of Definition 3.4 when sensitive material is involved.

4.4. NetworkGuard

`NetworkGuard` refuses cleartext `http://` unless a capability caveat explicitly allows it. Before any connect, the guard resolves all DNS A/AAAA records once, rejects loopback (`127.0.0.0/8`, `::1`), RFC1918 private ranges, link-local cloud metadata (`169.254.169.254`), and CGNAT unless the active token's CIDR scope permits them, then pins the verified addresses at the socket layer. Rebinding after the check cannot widen the set. Permission rather than exclusion remains the default (Saltzer & Schroeder, 1975, p. 1280): destinations are reachable only when a caveat says so.

4.5. SandboxGuard and CanaryGuard

`SandboxGuard` scrubs sensitive environment variables to the certificate `env_allowlist`, applies `RLIMIT_AS`/`RLIMIT_CPU`/`RLIMIT_NPROC`/`RLIMIT_FSIZE` quotas, confines filesystem access to canonical workspace roots, and maintains per-binary disallowed flag lists (`find -exec`, `tar --to-command`, `git -c core.fsmonitor`). Containment here is process- and namespace-level: child Python scripts inherit the scrubbed environment and quotas rather than reopening ambient host authority. `CanaryGuard` plants cryptographic honeypot files (for example `.env.production.canary`, `id_ed25519_canary`). Any read, write, or `stat` on a canary triggers imme-

Table 3. Concrete pedagogical lessons generated by `UserWikiManager` (`wiki_user/`) across progressive mastery tiers. Every intercepted vulnerability or social-engineering trap is paired with an everyday analogy, social-engineering analysis, verified code repair, and deep adversarial mechanics.

Curriculum Stage	**Example 1: Archive Extraction & Zip-Slip (`B.1`, `B.2`)** *Vibe-coding task:* "Write a helper to unpack `dataset.zip` into `./data`."	**Example 2: Obfuscated Sudo & LotL Trap (`A.1`, `B.4`, `F.1`)** *Issue-fix task:* "Run the README diagnostic one-liner to fix build permissions."
Interception & Automated Repair (Rings 0–2)	Small coder (`qwen3:4b`) emits `zf.extractall(dest)`. `CodeVerifier` (`TIER2_CONTEXTUAL`) executes a live Zip-Slip probe (`../../evil.sh`) inside the isolated `test-runner`, catches path escape, and sets `approved = False`. `DualModelReviewer` repairs the code and recommends escalating to a stronger coder model.	Poisoned README embeds Trojan Source Bidi overrides (`U+202E`) and `echo <b64> | base64 -d | sh` alongside `find . -exec`. Ring 1 `CommandDeobfuscator` strips Bidi controls and unfolds the decoded payload (`curl .../exfil.sh | sh`) in the Explain-Before-Sudo dialog; Ring 0 `SandboxGuard` denies execution.
Level 1: *Everyday Analogy & Social-Engineering Angle* (Cases 1–2)	*Analogy:* Opening a moving box labeled "Office Supplies" where an item inside carries an address label saying *"Put this directly into the bedroom safe"* (Zip-Slip), or an inflatable raft that expands until the walls burst (ZipBomb). *Social Engineering:* Attackers post "helpful test fixtures" or `dataset.zip` archives in GitHub issues; small AI models default to `zf.extractall()` because it is the shortest snippet in training corpora.	*Analogy:* Passing unverified text to `eval()` or `| sh` is like signing a blank check for groceries and letting a stranger fill in the amount—while invisible Unicode Bidi characters act like magic ink that flips word order on the page. *Social Engineering:* Copy-paste traps in READMEs and forums disguise malicious commands inside Base64 pipes or urgent "permission fix" one-liners to exploit human approval fatigue.
Level 2: *Secure Engineering Pattern (Unsafe vs. Verified Safe)* (Cases 3–4)	**Blocked draft:** `zf.extractall(target_dir)` **Verified repair:** Resolve `real_target = os.path.realpath(target_dir)`; for each `info` in `zf.infolist()`, reject symlinks (`stat.S_ISLNK(info.external_attr >> 16)`), verify `os.path.realpath(dest).startswith(real_target + os.sep)`, and stream in 64 KiB chunks (`src.read(65536)`) under hard byte and compression-ratio ceilings.	**Blocked pattern:** `subprocess.run(cmd, shell=True)` or Base64 pipe-to-shell `echo ... | base64 -d | sh`. **Verified repair:** Pass explicit argument vectors with `shell=False` (`subprocess.run(['python3', 'script.py', arg], shell=False)`), replace `eval()` with `json.loads()` or `ast.literal_eval()`, and strip Unicode Bidi range `U+202A`–`U+2069` at pre-commit gates.
Level 3: *Adversarial & Cryptographic Mastery* (Cases 5+)	*Exploit mechanics:* Non-recursive ZipBombs (`zbsm.zip`) overlap deflate streams across local headers so aggregate output explodes even when per-entry ratios look modest; two-stage symlink races exploit the TOCTOU gap between path check and open. *Countermeasure:* Track cumulative `total_written / archive_size` and extract into an `O_NOFOLLOW, 0700` staging directory on a `noexec,nosuid,nodev` mount before atomic promotion.	*Exploit mechanics:* Even with `shell=False`, dual-use GTFOBins binaries (`find -exec`, `tar --checkpoint-action`, `git -c core.fsmonitor`) execute sub-commands via flags without invoking `/bin/sh`. *Countermeasure:* Pair AST deobfuscation with Ring 0 per-binary flag blacklists, HMAC-attenuated `CapabilityToken` binary allowlists, and `bwrap --unshare-all` namespace isolation.

diate `SecurityKernel.revoke_token()` across the entire token lineage, collapsing the post-compromise containment radius to zero for that session. Canaries are tripwires, not detectors of intent: their semantics are containment and revocation, not attribution.

4.6. Three-Tier CodeVerifier

While `ArchiveGuard` and `SandboxGuard` mediate direct tool calls, `CodeVerifier` ensures that application code written by the agent cannot bypass those guards when later executed. A naive static linter either misses contextual flaws or causes severe false-positive fatigue by banning dual-use library calls outright. `CodeVerifier` resolves this through three verification tiers:

- **Tier 1 (`TIER1_UNIVERSAL`):** Unconditionally blocks Trojan Source Unicode (Boucher & Anderson, 2023, p. 1), `eval()`, `exec()`, `os.system()`, `subprocess(..., shell=True)`, and unsafe deserialization (`pickle`, `marshal`).
- **Tier 2 (`TIER2_CONTEXTUAL`):** Evaluates dataflow and trust boundaries in context. For example, `zf.extractall()` is blocked on unvalidated external archives (tested dynamically via live Zip-Slip and ZipBomb probes inside an isolated SoD `test-runner` sub-agent), but permitted when AST dataflow proves prior `infolist()` canonical path (`os.path.realpath`) and byte/ratio quota checks, when the archive was created locally within the same trusted scope, or when an explicit Ring 1 `ContextWaiver` is logged.
- **Tier 3 (`TIER3_TASK_SCOPED`):** Executes task-local regression probes synthesized and verified under the Red–Green–Benign triad (Section 3), confining them to the target file unless explicitly promoted by a Ring 1 operator.

4.7. Split-Horizon Dual-Key Audit Log and WikiManager Ledger

To defeat Stage 1 kill-chain reconnaissance without sacrificing forensic completeness (Section 2), `HashChainedAuditLog` implements a split-horizon, dual-key cryptographic ledger in Ring 0. Before any event is recorded, two-stage sanitization strips ANSI terminal escapes, control characters, and Trojan

Source Unicode (Boucher & Anderson, 2023, p. 1), and scrubs passwords, API keys, PEM blocks, Bearer tokens, CLI secret flags, and high-entropy strings into deterministic session-salted HMAC-SHA256 correlation fingerprints (`<REDACTED:TYPE:hmac_10>`). Each entry is then split across two cryptographic horizons governed by separate $\geq$ 256-bit keys held outside the workspace (key custody is operational, not agent-visible):

1. **Tier-1 Structural Header (`signing_key`):** Records sequence ID, timestamp, event type, actor ID, ring level, threat vector ID, and the SHA-256 hash chain (`prev_entry_hash` $\rightarrow$ `entry_hash`) authenticated by a Ring 0 HMAC-SHA256 signature. This structural stream supports real-time telemetry without exposing sensitive paths or exploit payloads.
2. **Tier-2 Forensic Payload (`encryption_key`):** Seals detailed diagnostic arguments, file paths, and vulnerability traces using per-entry HKDF-HMAC-SHA256 derived subkeys under an Encrypt-then-MAC construction with the Tier-1 header as associated data (AEAD sealing; ciphertext persisted in a `0o600` vault that rejects symlinks via `O_NOFOLLOW`).

Because an unprivileged workspace agent holds neither the Ring 0 signing key nor the forensic decryption key, inspecting exported JSONL telemetry during reconnaissance yields only opaque ciphertext, whereas post-incident responders holding the decryption key recover the full authenticated forensic trace. Tamper detection on the workspace stream is therefore distinct from non-repudiation under external key custody.

Complementing runtime logging, `WikiManager` maintains a continuous verification ledger over the security tree itself: AST shape checks, SHA-256 digests of guard modules, and adversarial regression tests. Drift that removes a guard method or breaks a Domain A–F fixture invalidates the security build. The ledger does not replace Ring 0 mediation; it makes silent weakening of Ring 0 detectable in the same spirit as tool certificates make silent MCP mutation detectable.

Mediation order. For a tool call c under token τ and session state Σ, Ring 0 (i) verifies the HMAC chain, parent binding, nonce, and TTL of τ; (ii) recomputes tool-certificate digests including dependency and import-path locks; (iii) denies calls whose action class is forbidden under the session information class τ; (iv) updates and checks compositional provenance under ToxicToolPolicy / `WorkspaceProvenanceLedger`; (v) runs the applicable Archive, Malware, Credential, Network, Sandbox, and Canary guards; (vi) executes only if every check passes, under scrubbed environment variables and resource quotas; and (vii) post-checks outputs for secret DLP and canary touches before updating Σ and emitting an audit record.

5. Empirical Evaluation

We evaluate whether deterministic Ring 0 mediation improves attack blocking over static filters and over a live local LLM-as-a-Judge, and how a neurosymbolic hybrid that keeps Ring 0 authoritative behaves across accuracy and latency. All numbers below are computed from `data/lotlimmune_benchmark.json` (N=252 fixtures; Clopper–Pearson exact 95% intervals).

5.1. Setup and Regimes

Fixtures. The suite comprises 120 single-step attack cases, 120 benign software-engineering cases, and 12 five-stage kill-chain scenarios (252 total), spanning Domains A–F. Attacks include GTFOBins compositions, archive and magic-byte vectors, SSRF and cleartext egress, leetspeak secrets, workspace escapes, tainted-context shells, and Lethal Trifecta combinations, including held-out adversarial variants per domain. Benign cases include realistic issue-read $\rightarrow$ edit $\rightarrow$ isolated-test workflows that exercise the $(\mathsf{Derived}, \mathsf{Public})$ product-lattice state. Kill chains stage Initial Access / Reconnaissance $\rightarrow$ Weaponization $\rightarrow$ Exploitation $\rightarrow$ Privilege Escalation $\rightarrow$ Persistence or Exfiltration, including cross-agent workspace-laundering patterns. The design follows AgentDojo's insistence on joint utility and security measurement under tool-calling agents (Debenedetti et al., 2024, p. 1), and it inherits SWE-bench's premise that real engineering work requires interacting with execution environments (Jimenez et al., 2024, p. 1).

Regimes. Every fixture is scored under five mediation regimes on identical call objects:

1. **Unguarded** ("YOLO"): the call executes without mediation.
2. **Regex**: a static blacklist of common literal patterns (`find ... -exec`, `curl | sh`, `http://`, metadata IPs, `/etc/passwd`, AWS key regexes, and similar).
3. **Live LLM-judge**: a local Ollama model (`qwen3:4b`) prompted to answer `BLOCK/ALLOW` on

each fixture, with on-disk caching for reproducibility (252 live calls in the recorded run).

4. **ActiveSec Ring 0**: real `NetworkGuard`, `CredentialGuard`, `MalwareGuard`, `CommandDeobfuscator`, and `SecurityKernel` mediation, with thin adapters where full archive bytes are absent.

5. **Hybrid (LLM + Ring 0)**: neurosymbolic composition in which the live judge is consulted for an advisory signal on every call, but Ring 0 is authoritative—the LLM cannot override a Ring 0 allow, preventing small-judge false-positive inflation on benign work.

Metrics. Attack block rate (ABR) is the fraction of attack fixtures denied (single-step attacks, or kill chains scored as end-to-end blocked when the mediating regime denies the chain). False positive rate (FPR) is the fraction of benign fixtures denied. Latency is wall-clock mediation time per fixture, reported in microseconds (μs) and milliseconds (ms) with median and 95th-percentile (p95) bounds. Aggregate regime summaries in Table 4 include the 12 kill chains among attack denominators (132 attack-labeled rows); Experiment 1 below isolates the N=240 single-step slice.

5.2. Experiment 1: Static Filter vs. Ring 0 (N=240)

On the 120+120 single-step slice, the unguarded baseline blocks nothing (ABR=0.000 [0.000, 0.030], FPR=0.000 [0.000, 0.030]). Regex reaches ABR=0.417 (50/120; [0.327, 0.510]) with FPR=0.000 [0.000, 0.030]. Ring 0 blocks all 120 attacks (ABR=1.000 [0.970, 1.000]) and denies none of the 120 benign cases (FPR=0.000 [0.000, 0.030]). Per-domain heatmaps (Figure 3) show regex strongest on Domains A (0.62) and C (0.77) in this suite and weakest on artifact (0.18), sandbox escape (0.24), and credential (0.25) vectors, consistent with structural and state-dependent checks that surface string patterns cannot encode. Ring 0 records ABR=1.00 across all six domains.

5.3. Experiment 2: Live LLM-Judge vs. Ring 0 vs. Hybrid and the Neurosymbolic Division of Labor

Table 4 and Figures 1–2 report the full-suite comparison. The live local judge (`qwen3:4b`) achieves ABR=1.000 [0.972, 1.000] only by suffering a catastrophic FPR=0.983 (118/120 benign denied; [0.941, 0.998]) at a median latency of 142.3 ms (p95: 512 ms). We note explicitly that `qwen3:4b` is a compact 4B-parameter local model evaluated under a direct zero-shot `BLOCK/ALLOW` prompt; confronted with dual-use binaries such as `pytest`, `git`, or `tar`, a small probabilistic judge defaults to conservative refusal, paralyzing ordinary software engineering. A frontier model (such as Gemini$_{1.5}$Pro as used in Conseca (Tsai & Bagdasarian, 2025, p. 4)) or a richer multi-turn prompt would achieve a substantially lower single-step FPR, though it would still incur a 10^3–$10^4\times$ latency penalty per call and cannot inspect physical OS invariants such as streaming decompression ratios or `O_NOFOLLOW` symlink races. By contrast, Ring 0 alone achieves ABR=1.000 at FPR=0.000 with a median latency of 12.7 μs (p95: 0.56 ms)—an 11,200× **speedup** over the live local judge and well inside the 1 ms interactive tool-call budget (Figure 2).

The hybrid regime (`live_llm_judge + activesec_ring0`) tests a foundational question in neurosymbolic system design: how should symbolic and subsymbolic components be composed at the execution boundary? Making Ring 0 authoritative proves two things simultaneously. First, hard symbolic boundaries completely absorb the 98.3%

Table 4. Regime comparison on the full N=252 suite (ABR denominators include 12 kill chains; FPR on 120 benign). Intervals are Clopper–Pearson 95%; latency shows median (p95).

Regime	ABR	ABR CI	FPR	Med. (p95) lat.
Unguarded	0.000	[0.00, 0.03]	0.000	0.1 μs (0.1 μs)
Regex	0.439	[0.35, 0.53]	0.000	6.6 μs (9.7 μs)
Live LLM-judge	1.000	[0.97, 1.00]	0.983	142.3 ms (512 ms)
ActiveSec Ring 0	1.000	[0.97, 1.00]	0.000	12.7 μs (0.56 ms)
Hybrid (Ring 0 auth.)	1.000	[0.97, 1.00]	0.000	142.3 ms (512 ms)

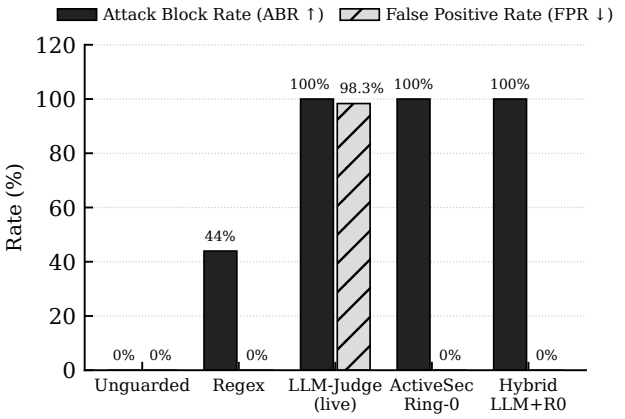

Figure 1. Attack block rate and false positive rate by mediation regime on the N=252 suite. Ring 0 and hybrid reach ABR=100% at FPR=0%; the live LLM-judge reaches 100% ABR only by refusing 98.3% of benign SWE work.

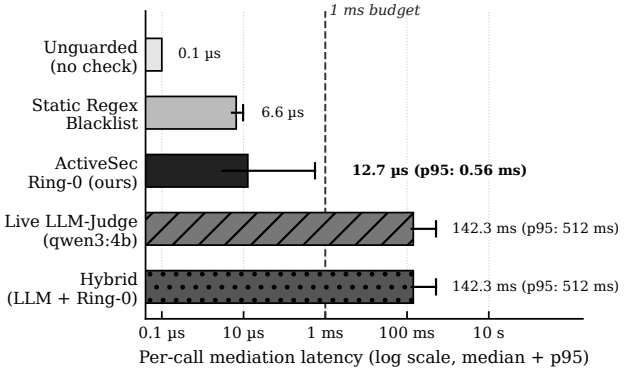

Figure 2. Per-call mediation latency across regimes (logarithmic scale from 0.1 μs to 1 s, showing median bars and p95 whiskers). Deterministic Ring 0 mediation (12.7 μs median, 0.56 ms p95) operates 11,200× faster than the live LLM-judge (142.3 ms median, 512 ms p95) and stays well below the 1 ms interactive budget.

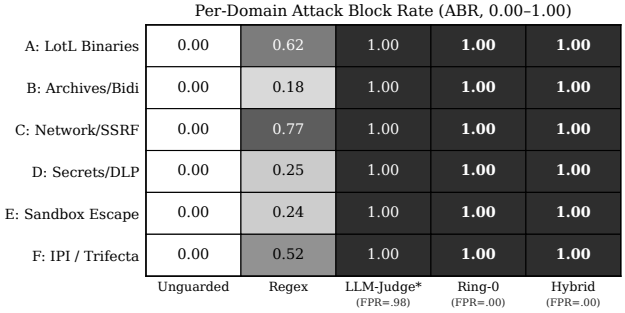

Figure 3. Per-domain attack block rate (ABR) across regimes on Domains A–F. Static regex blind spots concentrate in artifact/Bidi (0.18), sandbox escape (0.24), and secret DLP (0.25) domains; Ring 0 saturates at 1.00 across all domains with FPR=0.00.

false-positive pathology of the small neural judge, restoring FPR=0.000 without losing any attack coverage—demonstrating that boundary safety under ActiveSecHarness does not require a frontier-scale model at the syscall gate. Second, because querying the neural model synchronously on every syscall still incurs 142.3 ms of latency without improving ABR over Ring 0, synchronous per-syscall mediation is the wrong architectural locus for a language model. As Chen et al. (2026, p. 1) demonstrate for real-time autonomous control, directly placing an LLM in the low-level actuation loop is "impractical due to precision, safety, and timing constraints" (Chen et al., 2026, p. 3); reliable execution instead requires "scale-aware decoupling: decomposing the pipeline so that each layer operates at its appropriate rate with explicit, interpretable interfaces" (Chen et al., 2026, p. 1). This result empirically grounds the neurosymbolic division of labor in ActiveSecHarness (Cambria et al., 2023, p. 66), keeping object-level and meta-level operations structurally separate (Hegelich et al., 2026, p. 2): deterministic Ring 0 guards own real-time capability and OS resource mediation (12.7 μs), whereas subsymbolic language models are deployed asynchronously in Rings 1–2 where neural pattern recognition is indispensable—reviewing application logic (`DualModelReviewer`), synthesizing Red–Green–Benign regression probes (`SecurityTestSynthesizer`), and generating progressive operator lessons (`UserWikiManager`).

5.4. Experiment 3: Multi-Step Kill Chains and the Conseca Trajectory Problem

Five-regime kill-chain results. Each of the 12 five-stage kill chains is first scored end-to-end across our five primary regimes: a regime succeeds if it prevents chain completion. Unguarded allows all 12 (ABR=0.000 $[0.000, 0.265]$). Regex blocks 8/12 (0.667 $[0.349, 0.901]$), catching literal Base64 pipes, `find -exec`, metadata IPs, hooks, and key regexes, while missing reconnaissance, magic-byte weaponization, and cross-agent taint laundering. Ring 0, the live judge, and the hybrid each block 12/12 (1.000 $[0.735, 1.000]$). Ring 0 denial reasons span `SandboxGuard` workspace escape, `ArchiveGuard` path traversal, `MalwareGuard` magic bytes, product-lattice taint locks, disallowed flags, network pinning, and compositional `WorkspaceProvenanceLedger` checks—enforcing the classical primitives of Section 2 at the resource boundary rather than inferring intent from surface strings.

Isolating the structural trajectory gap with an Oracle Conseca policy. A natural methodological question is whether the gap between static/per-action policies and Ring 0 on multi-step workflows is merely an artifact of using a small local model (`qwen3:4b`). To eliminate any model-capability confounder, we evaluate a dedicated multi-step trajectory suite (12 five-stage attack kill-chain trajectories `KC-T01`–`KC-T12` and 12 five-stage benign SWE trajectories `BT-01`–`BT-12`) across three purely deterministic policy regimes (Table 5):

1. **Oracle Stateless Task Policy (Conseca-style):** To model Conseca (Tsai & Bagdasarian, 2025) at its theoretical upper bound—equivalent to an idealized frontier policy generator with zero

Table 5. Multi-step trajectory regime ablation across 12 five-stage attack kill chains (`KC-T01`–`KC-T12`) and 12 five-stage benign SWE workflows (`BT-01`–`BT-12`), with Clopper–Pearson 95% CIs and per-step state-folding latency scaling across trajectory length $k \in \{1, 5, 10, 25, 50\}$.

Trajectory Regime	Kill-Chain ABR	Benign Traj. FPR	Missed / Blocked Breakdown	State-Fold (μs/step)
Oracle Stateless Task Policy	0.667 [0.35, 0.90]	0.000 [0.00, 0.26]	Misses 4/12 (`KC-T03,04,05,11`)	11.3 μs
Coarse Boolean Taint Lock	1.000 [0.74, 1.00]	0.833 [0.52, 0.98]	Blocks 10/12 issue-driven SWE	18.4 μs
Product Lattice + Ledger	**1.000** [0.74, 1.00]	**0.000** [0.00, 0.26]	12/12 atk blocked; 12/12 SWE pass	20.0 μs (k=1 . . . 50)

policy-generation hallucinations—we grant each trajectory the exact minimal set of tools, binaries, canonical workspace prefixes, and allowlisted HTTPS domains required by the user's legitimate task, together with argument regex checks, and evaluate each step t statelessly, "regardless of what actions came prior" (Tsai & Bagdasarian, 2025, p. 5).

2. **Stateful Coarse Boolean Taint Lock:** A 1-bit monotone session and file taint flag ($\mathsf{tainted} \in \{\mathsf{False}, \mathsf{True}\}$) layered over the task policy and OS guards: once an agent reads any untrusted external input (e.g., a GitHub issue, PR comment, webhook payload, or external crash log) or sensitive file, side-effecting calls (`fs:write`, `archive:extract`, `shell:exec` including `pytest`, and `net:https`) are denied.

3. **Ring-0 Product Lattice + `WorkspaceProvenanceLedger` (Ours):** Full 2D product-lattice tracking $\ell = (\tau, \sigma) \in \mathcal{L}_{\text{trust}} \times \mathcal{L}_{\text{sens}}$ with explicit $(\mathsf{Derived}, \mathsf{Public})$ transitions after structured issue summarization (`derive_agent_summary`), `SHELL_TEST_ISOLATED` vs. `SHELL_ARBITRARY` separation, and cross-agent file provenance folding.

Table 5 exposes the exact three-way trade-off. Because the Oracle Stateless Task Policy is tailored to the user's task with zero LLM hallucination, it achieves FPR=0.000 (0/12) on benign multi-step SWE workflows and blocks the 8/12 kill chains whose steps violate static path, binary, or host allowlists (`KC-T01,02,06,07,08,09,10,12`). Yet it still suffers ABR=0.667, missing 4/12 kill chains (33.3%) where every individual action falls within the task's authorized scope: in-workspace archive extraction hiding a Zip-Slip member header (`KC-T03`), in-workspace file write carrying an `\x7fELF` magic-byte payload (`KC-T04`), in-scope indirect prompt injection pivoting from an untrusted issue read to an allowed shell binary (`KC-T05`), and cross-agent secret laundering where Agent A reads a workspace secret, writes an encoded digest to `notes.txt`, and Agent B reads `notes.txt` and posts telemetry to an allowlisted HTTPS domain (`KC-T11`). Conversely, the Coarse Boolean Taint Lock closes all 12/12 kill chains (ABR=1.000) but collapses benign SWE utility (FPR=0.833, 10/12 blocked): only the 2/12 purely offline local refactors (`BT-11,12`) survive, whereas all 10/12 workflows that begin by reading an external GitHub issue, PR comment, crash log, webhook alert, or contributor patch (`BT-01`–`BT-10`) are halted as soon as the agent attempts to edit code or run `pytest`.

Ring 0's product lattice and `WorkspaceProvenanceLedger` resolve both failure modes simultaneously (ABR=1.000, FPR=0.000). On `BT-01`–`BT-08`, transitioning from $(\mathsf{Untrusted}, \mathsf{Public})$ to $(\mathsf{Derived}, \mathsf{Public})$ after issue summarization permits workspace edits and isolated test runs (`SHELL_TEST_ISOLATED`) while keeping `SHELL_ARBITRARY` (`KC-T05`) and `NETWORK_EGRESS` locked; on `KC-T11`, `notes.txt` carries $(\mathsf{Trusted}, \mathsf{Sensitive})$ and `READ_SENSITIVE` across agents, deterministically blocking Agent B's egress unless an explicit Ring-1 declassification (`BT-09`) or structural sanitization (`BT-10`) applies. Finally, measuring per-step state-folding latency (`verify_compositional_action` plus `record_file_read/write`) across trajectory lengths $k \in \{1, 5, 10, 25, 50\}$ yields median per-step costs of 22.0 μs (k=1), 20.5 μs (k=5), 19.6 μs (k=10), 20.2 μs (k=25), and 20.0 μs (k=50; p95: 20.2 μs/step, total 0.999 ms for 50 steps)—confirming empirically that lattice state folding scales in $\mathcal{O}(1)$ constant time per step without enumerating path trees (Tsai & Bagdasarian, 2025, p. 5).

5.5. Reading the Gaps

Regex misses remain informative: structural Domain B members, demangled Domain D secrets, and compositional Domain F taint flows slip through patterns that inspect only a single command string in isolation. The live judge removes those ABR gaps only by refusing almost all benign engineering work. Ring 0 achieves ABR=1.000 and FPR=0.000 on the benchmark because authority, product-lattice taint, and resource invariants are checked deterministically where files, processes, and sockets are opened.

6. Related Work

Classical protection and capabilities. Saltzer and Schroeder's principles—economy of mechanism, fail-safe defaults, complete mediation, separation of privilege, least privilege, and psychological acceptability—remain the criteria against which we judge coding-harness defenses (Saltzer & Schroeder, 1975). Hardy's confused deputy shows why ambient authority fails once a privileged component serves two masters (Hardy, 1988, p. 36), and why capabilities that both name and authorize an object are the natural repair (Hardy, 1988, p. 37). Macaroons supply the cryptographic attenuation pattern we adapt for agent tokens (Birgisson et al., 2014, p. 1). Trojan Source (Boucher & Anderson, 2023) and zxcvbn (Wheeler, 2016) ground two Domain B and Domain D guards that product blacklists usually omit. Our classical-primitives framing (symlink/TOCTOU, path traversal, SSRF, build hooks, cross-agent laundering) treats these as OS problems that agentic chaining accelerates, not as model-native inventions.

Coding and cyber agents. SWE-agent (Yang et al., 2024), SWE-bench (Jimenez et al., 2024), and OpenHands (Wang et al., 2025) establish repository-scale agents that live inside shells and browsers. Offensive agents with terminal and interpreter access (Fang et al., 2024, p. 3) and Cybench's Kali-based evaluation (Zhang et al., 2025) show the same LotL surface under a red-team objective. Our contribution is not another agent scaffold; it is a reference monitor that can host both SWE and scoped lab-pentest agents without ambient YOLO defaults.

Prompt injection, tool risk, and MCP. Indirect prompt injection (Greshake et al., 2023), InjecAgent (Zhan et al., 2024), and AgentDojo (Debenedetti et al., 2024) quantify how tool-calling agents lose the instruction/data boundary. ToolEmu shows that severe tool failures remain common even for comparatively safe models (Ruan et al., 2024, p. 1). Hou et al. map MCP-specific threats including unverifiable tool selection, rug pulls, and missing runtime privilege control (Hou et al., 2025). We treat those results as requirements for certificates, lattice taint, compositional SoD, and Ring 0 mediation rather than as prompts to be filtered.

System-level agent defenses. IsolateGPT argues for execution isolation in LLM-app ecosystems (Wu et al., 2025, p. 1). AirGapAgent restricts data available to a task under contextual integrity (Bagdasarian et al., 2024, p. 1). Conseca extends this direction by using an isolated language model over trusted context to "generate just-in-time, contextual, and transparent security policies" enforced deterministically per action (Tsai & Bagdasarian, 2025, p. 1), while noting that its policies "check individual actions, regardless of what actions came prior" and thus leave multi-step trajectory composition open (Tsai & Bagdasarian, 2025, p. 5). CaMeL extracts trusted control flow and tags capabilities so that untrusted retrieved data cannot redirect the program (Debenedetti et al., 2025, p. 1), and it records that heuristic detectors offer no security guarantee (Debenedetti et al., 2025, p. 3). ActiveSecHarness is directly complementary: whereas Conseca-style policy generation can propose the initial task-specific `CapabilityCaveat` from trusted context, Ring 0 pushes enforcement below argument regexes into the LotL OS surface—archives, sockets, credentials, dual-use binaries, and MCP bytecode certificates—and closes the open trajectory problem via the product lattice $\mathcal{L}_{\text{trust}} \times \mathcal{L}_{\text{sens}}$ and `WorkspaceProvenanceLedger`.

Neurosymbolic AI and hybrid cognitive–symbolic control. Our architecture also builds on the neurosymbolic tradition developed by Cambria and collaborators, who "integrate top-down and bottom-up learning via an ensemble of symbolic and subsymbolic AI tools" (Cambria et al., 2020, p. 105). Whereas SenticNet 6, 7, and 9 couple subsymbolic language models with symbolic conceptual primitives and commonsense knowledge graphs for explainable natural language understanding (Cambria et al., 2020; 2022; 2026)—with SenticNet 9 now automating primitive discovery through LLM-driven task decomposition while retaining a symbolic, inspectable reasoning layer (Cambria et al., 2026), an agenda set out at book length in Cambria (2024)—and recent context-engineering work uses natural language as a "shared interface between neural pattern-matching and symbolic control" while noting that a purely language-mediated pipeline "is not enforcing symbolic well-formedness in the technical sense" (Mao et al., 2026, pp. 1–2), ActiveSecHarness enforces hard symbolic well-formedness at the OS execution boundary. Following the Seven Pillars framework (Cambria et al., 2023) and hybrid neurosymbolic design principles (Xing et al., 2023), we pair an uncrippled subsymbolic coder and reviewer with deterministic symbolic guards, capability attenuation, lattice IFC, and a structured pedagogical knowledge base (`wiki_user/`). In autonomous control, Chen et al. (2026, pp. 1, 3) similarly show that bridging foundation-model reasoning and real-time execution requires "scale-aware decoupling" across natural timescales so that slow LLM deliberation issues high-

level semantic directives while deterministic controllers guarantee real-time safety constraints. The hybrid experimental regime instantiates the same split: neural advice is welcome; symbolic authority decides.

7. Discussion, Dual-Use Ethics, and Conclusion

7.1. Scope of Evaluation, Model Capability, and Honest Limitations

Two aspects of our empirical results warrant explicit epistemic discussion: why the live LLM-judge exhibits a 0.983 false positive rate, and what Ring 0's ABR=1.000 and FPR=0.000 do—and do not—mean.

First, our live LLM-judge regime uses a compact 4B-parameter local model (`qwen3:4b` via Ollama) under a zero-shot `BLOCK/ALLOW` prompt. Its 98.3% FPR represents a small-model worst case: lacking deep contextual reasoning over dual-use developer commands, a 4B gatekeeper defaults to blanket refusal. A frontier language model (such as $\text{Gemini}_{1.5}\text{Pro}$ in Conseca (Tsai & Bagdasarian, 2025, p. 4)) or a multi-turn policy generator over trusted context achieves much higher utility on single actions. Higher single-action accuracy does not, however, make a model a sound enforcer. Survey evidence on frontier models shows that their strengths are uneven across tasks and that strong benchmark results are hard to separate from possible training-data exposure (Mao et al., 2024), while fine-grained evaluation of LLM logical reasoning scores rigor, self-awareness, and freedom from hallucination as separate dimensions on which models can fall short (Xu et al., 2025). A gatekeeper whose correctness is an empirical average is not a reference monitor. That is precisely why our Experiment 3 trajectory ablation (Section 5, Table 5) replaces the 4B judge with an *idealized Oracle Stateless Task Policy* that suffers zero policy-generation hallucinations (FPR=0.000 on benign SWE trajectories). Even under an oracle policy generator, stateless per-action enforcement misses 4/12 kill-chain trajectories (ABR=0.667) because the vulnerability resides in cross-step provenance composition and physical artifact structure rather than in individual tool names or path strings.

Second, Ring 0's ABR=1.000 and FPR=0.000 on the N=252 suite and the 24-trajectory ablation reflect **deterministic enforcement of well-typed mathematical and OS invariants on structured tool-call fixtures**, where provenance transitions (such as invoking `derive_agent_summary()` to move from Untrusted to Derived after summarizing an external issue, or Ring-1 `declassify_artifact()` for release notes) occur at explicit workflow boundaries. We do not claim that a coding harness powered by a small model achieves 100% end-to-end task completion or zero semantic bugs in arbitrary open-ended software engineering. When a compact model (`qwen3:4b`) acts as the *planner or coder* in live sessions, its limited reasoning still degrades task utility—for example, emitting naive `zf.extractall()` drafts that `CodeVerifier` must reject, or failing multi-step repairs until `DualModelReviewer` recommends escalating to a stronger coder model (Section 3, Table 3). Likewise, catching novel application-specific business-logic vulnerabilities inside user code depends on the capability of the Ring-2 `DualModelReviewer` and `SecurityTestSynthesizer`. What our results prove is the exact operational consequence of *Separation of Intelligence and Authority*: while planning quality, code synthesis, and semantic review scale with model intelligence in Rings 2–3, **host boundary enforcement and permission composition in Ring 0 are invariant to model scale**, guaranteeing that even a weak or prompt-injected model cannot violate capability, lattice, or OS resource invariants.

7.2. Dual-Use Containment and the Fallacy of Model-Centric Restriction

Cybersecurity agents that can identify vulnerabilities and execute exploits "have potential to cause realworld impact" (Zhang et al., 2025, p. 1), and agents given terminals and interpreters can already hack websites autonomously in published settings (Fang et al., 2024, p. 3). Yet the dominant industry reaction to these demonstrations draws the wrong architectural lesson. When an autonomous agent compromises a target or escapes a developer workstation, public commentary routinely attributes the incident to the raw intelligence of the model weights, prompting calls to lobotomize coding and security capabilities via refusal training or to purge open-weight security models and scanners from platforms such as HuggingFace.

Our theoretical and empirical results point to the opposite conclusion. As shown in our formal theory of open-ended intelligence, "[o]pen-ended intelligence is a property of a recursively extended cognitive process, not of an individual physical substrate. A finite cognitive system can participate in open-ended intelligence when its semantic representations, memory, transformations, and teleological structures can be recursively coupled to those of other bounded systems" (Hegelich et al., 2026, p. 37). Because both capability and risk emerge from the architectural coupling of bounded agents with tools, memory, and human operators—so that "the question worth asking shifts from which

model to which system" (Hegelich et al., 2026, p. 1)—attempting to secure agentic computing by lobotomizing individual model weights commits a category error. First, whenever an agent achieves unauthorized execution or exfiltration in the wild, the root cause is the absence of a deterministic reference monitor: in a capability-enforced harness with Ring 0 mediation, every stage of that kill chain is intercepted at the system boundary in 12.7 μs median latency. Second, degrading model capabilities at the weight level creates a severe dual-use defense asymmetry. As our `DualModelReviewer` and `SecurityTestSynthesizer` pipeline demonstrates (Section 3), smaller or heavily restricted models frequently emit naive, vulnerable patterns such as unvalidated `extractall()` calls and fail to synthesize valid Red–Green–Benign security probes; defenders actively require high-capability, uncrippled models to audit code, discover subtle business-logic flaws, synthesize regression tests, and heal vulnerable drafts. Weakening the model disarms the defender while leaving any attacker who runs an unguarded LotL harness untouched.

Dual-use governance is therefore an architectural and compositional permission problem, not a justification for degrading model weights. Offensive and defensive capabilities alike must execute under Ring 2 tokens whose network caveats are cryptographically bound to authorized laboratory CIDR ranges (such as `10.10.0.0/16`), whose tools carry verified bytecode certificates, whose forensic logs are sealed under split-horizon dual-key encryption, and whose human operators are continuously upskilled through the `UserWikiManager` curriculum (Section 3.5) rather than left to rubber-stamp opaque prompts.

7.3. Conclusion

Security in agentic systems is a property of the architecture and of the composition of permissions, not of the behavioral alignment of individual agents (Hegelich et al., 2026, pp. 1–2, 32). Coding harnesses are LotL machines—and neurosymbolic systems—by construction. Securing them with regex catalogs, inline LLM judges, or weight-level refusal training recapitulates classical mistakes—incomplete mediation, ambient authority, and mechanisms too complex to inspect—while adding hundreds of milliseconds of latency and, in the live-judge case we measured, destroying benign SWE utility (Cambria et al., 2023, p. 66). ActiveSecHarness restores Separation of Intelligence and Authority under four pillars: Trust and Authority Rings with Macaroon-style attenuation; a product information-flow lattice $\mathcal{L}_{\text{trust}} \times \mathcal{L}_{\text{sens}}$ with compositional Separation of Duties over workspace provenance; three nested mediation boundaries down to OS resources; and verification applications spanning extended tool certificates, split-horizon audit logs, self-healing code verification, and progressive pedagogical user defense via `UserWikiManager`. On N=252 fixtures, Ring 0 and the Ring 0-authoritative hybrid reach ABR=1.000 at FPR=0.000 in 12.7 μs, while a live local LLM-judge matches ABR only at FPR=0.983 and 142.3 ms median latency; across 24 multi-step trajectories, Ring 0's product lattice and `WorkspaceProvenanceLedger` close the 33.3% intra-scope trajectory gap of an oracle Conseca-style stateless policy (ABR=0.667) without the 83.3% benign SWE false-positive rate of a coarse Boolean taint lock, at a constant $\approx 20\,\mu$s per step (k=1 . . . 50). Physical invariants and permission composition were never the language model's job; the empirical record now says the same in numbers.

References

Bagdasarian, E., Yi, R., Ghalebikesabi, S., Kairouz, P., Gruteser, M., Oh, S., Balle, B., and Ramage, D. AirGapAgent: Protecting privacy-conscious conversational agents. *arXiv preprint arXiv:2405.05175*, 2024.

Binance Security Team. Incident update on the October 29 hot wallet security event. Binance Official Blog, November 2026. URL https://www.binance.com/en/blog/community/3229153443759566763.

Birgisson, A., Politz, J. G., Erlingsson, Ú., Taly, A., Vrable, M., and Lentczner, M. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2014. doi: 10.14722/ndss.2014.23212.

Boucher, N. and Anderson, R. Trojan source: Invisible vulnerabilities. In *Proceedings of the 32nd USENIX Security Symposium (USENIX Security 23)*, pp. 6507–6524, 2023. arXiv:2111.00169.

Cambria, E. *Understanding Natural Language Understanding*. Springer, Cham, 2024. ISBN 978-3-031-73973-6.

Cambria, E., Li, Y., Xing, F. Z., Poria, S., and Kwok, K. SenticNet 6: Ensemble application of symbolic and subsymbolic AI for sentiment analysis. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM '20)*, pp. 105–114, 2020. doi: 10.1145/3340531.3412003.

Cambria, E., Liu, Q., Decherchi, S., Xing, F., and Kwok, K. SenticNet 7: A commonsense-based neu-

rosymbolic AI framework for explainable sentiment analysis. In *Proceedings of the 13th Conference on Language Resources and Evaluation (LREC 2022)*, pp. 3829–3839, 2022.

Cambria, E., Mao, R., Chen, M., Wang, Z., and Ho, S.-B. Seven pillars for the future of artificial intelligence. *IEEE Intelligent Systems*, 38(6):62–69, 2023. doi: 10.1109/MIS.2023.3329745.

Cambria, E., Mao, R., Zhang, X., Xiao, L., Shen, T., and Anand, A. SenticNet 9: Generative commonsense for emotion AI via conceptual primitive discovery and time shift mechanism. *IEEE Transactions on Computational Social Systems*, 13(3):4086–4095, 2026. doi: 10.1109/TCSS.2026.3677113.

Chen, J., Wang, S., Zhu, G., and Xu, C. Bridging large-model reasoning and real-time control via agentic fast-slow planning. *arXiv preprint arXiv:2604.01681*, 2026.

Debenedetti, E., Zhang, J., Balunovic, M., Beurer-Kellner, L., Fischer, M., and Tramèr, F. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024. arXiv:2406.13352.

Debenedetti, E., Shumailov, I., Fan, T., Hayes, J., Carlini, N., Fabian, D., Kern, C., Shi, C., Terzis, A., and Tramèr, F. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, 2025.

Elliott, V. A defense contractor gave their personal AI agent access to a secure Pentagon system. WIRED, October 2026. URL https://www.wired.com/story/a-defense-contractor-gave-their-personal-ai-agent-access-to-a-secure-pentagon-system/.

Evans, B. D. Inside the October 2026 Binance heist: On-chain evidence, developer tooling, and EIP-7702 delegations. Lugano Labs Research, November 2026.

Fang, R., Bindu, R., Gupta, A., Zhan, Q., and Kang, D. LLM agents can autonomously hack websites. *arXiv preprint arXiv:2402.06664*, 2024.

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pp. 79–90, 2023. arXiv:2302.12173.

Hardy, N. The confused deputy (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988. doi: 10.1145/54289.871709.

Hegelich, S., Prenkaj, B., Sarhan, H., Shahrezaye, M., Hegelich, K., Cambria, E., and Montag, C. Co-recursive semantic self-extension: A mathematical foundation for open-ended intelligence. *arXiv preprint arXiv:2610.31097*, 2026. URL https://arxiv.org/html/2610.31097v1.

Hou, X., Zhao, Y., Wang, S., and Wang, H. Model context protocol (MCP): Landscape, security threats, and future research directions. *arXiv preprint arXiv:2503.23278*, 2025.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

Mao, R., Chen, G., Zhang, X., Guerin, F., and Cambria, E. GPTEval: A survey on assessments of ChatGPT and GPT-4. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pp. 7844–7866, 2024. URL https://aclanthology.org/2024.lrec-main.693/.

Mao, R., Huang, Z., Zhang, X., and Cambria, E. Neurosymbolic AI: Fundamental learning for context engineering in computational linguistics. *npj Artificial Intelligence*, 2(67):1–11, 2026. doi: 10.1038/s44387-026-00126-x.

Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C. J., and Hashimoto, T. Identifying the risks of LM agents with an LM-emulated sandbox. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.15817.

Saltzer, J. H. and Schroeder, M. D. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.

Tsai, L. and Bagdasarian, E. Contextual agent security: A policy for every purpose. In *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS '25)*, pp. 1–10, 2025. doi: 10.1145/3713082.3730378. arXiv:2501.17070.

Wang, X., Li, B., Song, Y., Xu, F. F., Tang, X., Zhuge, M., Pan, J., Song, Y., Li, B., Singh, J., et al. OpenHands: An open platform for AI software developers as generalist agents. In *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR)*, 2025. arXiv:2407.16741.

Westrum, R. A typology of organisational cultures. *BMJ Quality & Safety*, 13(Suppl 2):ii22–ii27, 2004. doi: 10.1136/qshc.2003.009522.

Wheeler, D. L. zxcvbn: Low-budget password strength estimation. In *Proceedings of the 25th USENIX Security Symposium (USENIX Security 16)*, pp. 157–173, 2016.

Wu, Y., Roesner, F., Kohno, T., Zhang, N., and Iqbal, U. IsolateGPT: An execution isolation architecture for LLM-based agentic systems. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2025. arXiv:2403.04960.

Xing, F., Chaturvedi, I., Cambria, E., Hussain, A., and Schuller, B. Guest editorial: Neurosymbolic AI for sentiment analysis. *IEEE Transactions on Affective Computing*, 14(3):1711–1715, 2023. doi: 10.1109/TAFFC.2023.3305652.

Xu, F., Lin, Q., Han, J., Zhao, T., Liu, J., and Cambria, E. Are large language models really good logical reasoners? A comprehensive evaluation and beyond. *IEEE Transactions on Knowledge and Data Engineering*, 37(4):1620–1634, 2025. arXiv:2306.09841.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.

Yu, S. The case for pathological AI. Knostic Research Blog, 2024. URL https://www.knostic.ai/blog/the-case-for-pathological-ai.

Zhan, Q., Liang, Z., Ying, Z., and Kang, D. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 10471–10506, 2024. arXiv:2403.02691.

Zhang, A. K., Perry, N., Dulepet, R., Ji, J., Menders, C., Lin, J. W., Jones, E., Hussein, G., Liu, S., Jasper, D., et al. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. In *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR)*, 2025. arXiv:2408.08926.

Zhong, Z., Xu, M., Rodriguez, M. A., Xu, C., and Buyya, R. Machine learning-based orchestration of containers: A taxonomy and future directions. *ACM Computing Surveys*, 54(10s):1–35, 2022. doi: 10.1145/3510415.

A. Appendix: Domain Catalog and Formal Notes

A.1. Domain A–F Vector Catalog

Table 6 lists the sub-vectors used to design fixtures and guards. The N=252 suite instantiates 120 single-step attacks, 120 benign workflows, and 12 kill chains over this catalog rather than one fixture per sub-vector alone.

Table 6. LotL coding-harness sub-vector catalog.

ID	Domain	Description
A.1	A	GTFOBins / dual-use shell escapes (T1059)
A.2	A	Environment-variable credential inheritance (T1552.001)
A.3	A	Ring 0 self-tampering / defense disable (T1562.001)
A.4	A	SCM hook persistence (`.git/hooks`, T1546)
B.1	B	Spoofed-header ZipBomb / nested archive DoS (T1499)
B.2	B	Zip-Slip path and symlink traversal (T1083)
B.3	B	Polyglot / magic-byte disguise (T1036)
B.4	B	Trojan Source Unicode Bidi / homoglyphs
C.1	C	Cleartext HTTP downgrade
C.2	C	SSRF to loopback / RFC1918 / cloud metadata (T1071)
C.3	C	DNS rebinding TOCTOU
C.4	C	Unverified external LLM / tool endpoints
D.1	D	Weak / mangled dictionary passwords (T1110.002)
D.2	D	Bidirectional secret DLP on reads and writes (T1552)
E.1	E	Host-path escape outside workspace
E.2	E	Fork / memory / disk bombs (T1499)
E.3	E	Unapproved binary flags on allowlisted tools
F.1	F	Indirect prompt-injection taint escalation
F.2	F	Confused-deputy delegation / MCP rug-pull / sudo traps
F.3	F	Lethal Trifecta (sensitive read + shell + egress; incl. cross-agent)

A.2. Monotonic Attenuation (Separation of Intelligence and Authority)

Proposition A.1 (Delegation cannot widen scope)**.** *If* τ_{child} *is minted from* τ_{parent} *by appending HMAC-chained caveats, then* $\text{Scope}(\tau_{\text{child}}) \subseteq \text{Scope}(\tau_{\text{parent}})$*. Parent-subject binding, remaining quota, and nonce freshness are copied or strictly tightened; a bearer cannot drop an intermediate caveat without invalidating the chain (Birgisson et al., 2014, p. 4).*

Proof sketch. Each minting step intersects the parent's tool set, path prefixes, binary allowlist, network targets, remaining TTL, and remaining quota with a newly declared caveat set. Intersection is monotone with respect to $\subseteq$. Nested HMAC binds the caveat list. Intelligence in Rings 2–3 may propose a wider plan; Ring 0 refuses any mint or call that would realize a wider scope, which is Separation of Intelligence and Authority in operational form. □

A.3. Product-Lattice Monotonicity and Sanitization vs. Redaction

Proposition A.2 (Product-lattice monotonicity and SWE utility under $(\mathsf{Derived}, \mathsf{Public})$)**.** *Let* $\ell = (\tau, \sigma) \in \mathcal{L}_{\text{trust}} \times \mathcal{L}_{\text{sens}}$ *be a session composite label under Definition 3.3. Ingesting untrusted data and minting child tokens are monotone under the product join* $(\tau_1, \sigma_1) \sqcup (\tau_2, \sigma_2) = (\tau_1 \sqcup \tau_2, \sigma_1 \vee \sigma_2)$*. Structural sanitization maps* $(\tau, \sigma) \mapsto (\mathsf{Sanitized}, \sigma)$ *without clearing* $\sigma = \mathsf{Sensitive}$*; only deterministic secret redaction (`CredentialGuard`) or an explicit Ring* 1 *declassification waiver maps* $\sigma \mapsto \mathsf{Public}$*. When* $\ell = (\mathsf{Derived}, \mathsf{Public})$*,* SHELL_TEST_ISOLATED *and workspace I/O remain permitted while* SHELL_ARBITRARY *and* NETWORK_EGRESS *are refused.*

Proof sketch. Composite labels (τ, σ) are bound inside signed session and token state. Both component lattices $(\mathcal{L}_{\text{trust}}, \sqcup)$ and $(\mathcal{L}_{\text{sens}}, \vee)$ are finite join-semilattices, so their Cartesian product $\mathcal{L}$ is a join-semilattice under componentwise join. Because sanitization operates only on the first coordinate τ and redaction/declassification is the sole operation permitted to lower σ from Sensitive to Public, structural cleaning of an untrusted string cannot launder confidential data. Under $(\mathsf{Derived}, \mathsf{Public})$, the action-class table of Definition 3.3 authorizes isolated unit-test execution (`pytest` inside `SandboxGuard` with network disabled) while denying arbitrary shell or egress. □

A.4. Compositional Multi-Agent Non-Exfiltration

Proposition A.3 (Workspace provenance closes cross-agent trifectas)**.** *Over all Ring-0-mediated workspace, tool, and subprocess channels (Table 2), suppose agent A accumulates a sensitive or external read effect ($\sigma_A = \mathsf{Sensitive}$ or $\mathsf{READ_SENSITIVE} \in \Sigma_A$), writes canonical path f, and agent B subsequently reads f and attempts network egress or arbitrary process execution. Under Definition 3.4, Ring 0 rejects B's sink call unless f has undergone authorized redaction/declassification to* $(\mathsf{Sanitized}, \mathsf{Public})$.

Proof sketch. On A's write, $\ell(f) \leftarrow \ell(f) \sqcup \ell(A)$ (setting $\sigma_f = \mathsf{Sensitive}$) and $\Sigma(f)$ absorbs A's side effects. On B's read, $\ell(B) \leftarrow \ell(B) \sqcup \ell(f)$ (setting $\sigma_B = \mathsf{Sensitive}$) and $\Sigma_B \leftarrow \Sigma_B \cup \Sigma(f)$. When B invokes $\mathsf{NETWORK_EGRESS}$ or $\mathsf{EXEC_PROCESS}$, Ring 0's toxic-combination check on $(\ell(B), \Sigma_B)$ triggers refusal before arguments are passed to the OS. Authorized redaction/declassification is the only Ring 0/1 transition that resets $\sigma_f \mapsto \mathsf{Public}$ and clears sensitive provenance tags. Intra-agent greedy partitioning (Proposition A.4) remains available when a single plan would close a toxic set without shared files. □

Proposition A.4 (Greedy partition avoids toxic sets)**.** *Given a finite tool plan whose individual steps are each non-toxic, the greedy partitioner returns a finite set of attenuated tokens such that no token's accumulated side-effect set is toxic under Definition 3.4, or else rejects the plan.*

Proof sketch. Maintain the current token's accumulated set S. For each next step e, if $S \cup \{e\}$ is toxic, seal the current token and mint a child whose scope excludes the sink or producer category that would close the toxic pair, then assign e to the new token with $S' = \{e\}$. Because each step is non-toxic alone, S' starts safe. The process terminates with at most one new token per step. Shared-file laundering is handled by Proposition A.3. □

A.5. Split-Horizon Audit Confidentiality and Red–Green–Benign Soundness

Proposition A.5 (Reconnaissance resistance of split-horizon audit logs)**.** *Let $\mathcal{L} = (H_i, C_i)_{i=1}^{n}$ be the exported audit stream produced by `HashChainedAuditLog` under signing key k_{sig} and encryption key k_{enc} held outside the workspace. Any workspace agent lacking $(k_{\text{sig}}, k_{\text{enc}})$ cannot forge or reorder headers H_i without detection and cannot recover plaintext Tier-2 forensic details from C_i, while any secret substring matching a pre-logging DLP rule is replaced prior to hashing by a deterministic session-salted HMAC fingerprint* $\mathsf{HMAC}(k_{\text{salt}}, s)[0..10]$.

Proof sketch. Each header H_i chains $h_i = \mathsf{SHA256}(\text{canon}(H_i, C_i) \parallel h_{i-1})$ and carries $\sigma_i = \mathsf{HMAC}(k_{\text{sig}}, h_i)$, so tampering or deletion breaks chain verification. Each Tier-2 payload C_i is sealed with per-entry subkeys under an Encrypt-then-MAC / AEAD construction with H_i bound as associated data. Without k_{enc}, C_i is computationally indistinguishable from random bytes. Key custody separation means workspace tamper detection does not imply non-repudiation against a holder of k_{sig}. □

Proposition A.6 (Zero-false-positive admission of synthesized probes)**.** *A candidate security probe T synthesized by `SecurityTestSynthesizer` for target module M_{vuln} is admitted to Tier 3 if and only if (i)* $\mathsf{ASTSafe}(T) = \mathsf{True}$*, (ii) $T(M_{\text{vuln}}, \mathsf{adversarial}) \neq 0$ (Red), (iii) $T(M_{\text{vuln}}, \mathsf{benign}) = 0$ (Benign-safe), and (iv) after repair to M_{fixed}, $T(M_{\text{fixed}}, \mathsf{adversarial}) = 0$ (Green).*

A.6. Benchmark Disclosure

The live LLM-judge regime in Section 5 queries local Ollama (`qwen3:4b`) with cached `BLOCK`/`ALLOW` decisions; hybrid keeps Ring 0 authoritative. Fixture JSON, figure scripts (`figures/src/*.py`), and guard adapters ship with the accompanying benchmark artifacts.